UNIVERSIDAD NACIONAL DE ASUNCION

FACULTAD DE INGENIERIA

C Á T E D R A D E C O M P U T A C I O N

MATERIAL DE REFERENCIA

Introducción a la Programación en Lenguaje C++

Segundo Ciclo - Año 2011

Versión 1.0

Autores:

***Barreto S. Néstor*, Prof. Ing.**

***Gonzalez V., Elizardo*, Prof. Ing.**

***Ovelar O., Juan*, Ing.**

***Rojas S., Héctor*, Prof. Ing.**

***Velazquez G., Ignacio*, Ing.**

Introducción

Este material editado por la cátedra de computación de la Facultad de Ingeniería de la Universidad Nacional de Asunción, contiene los capítulos que serán desarrollados en las distintas secciones durante el ciclo, los autores agradecen el permanente apoyo brindado por las autoridades de la Facultad de Ingeniería.

El lenguaje C++ es ideal para aprender la programación de computadoras al ser un lenguaje muy compacto con una sintaxis sencilla y fácil de aprender. Es también un lenguaje muy potente ya que se utiliza mucho en programación en todos los niveles, desde controladores de dispositivos y componentes de sistemas operativos hasta aplicaciones a gran escala. Existen compiladores de C para cualquier entorno de programación y de sistemas operativos, tales como Unix, Linux, Windows, Mac, etc., de modo que cuando el estudiante haya aprendido C estará en condiciones de programar en cualquier contexto y entorno actual. Lo aprendido es una base para luego continuar en programación orientada a objetos, Java o C#, todo en función del ambiente profesional en el cual se desenvolverá.

C++ es un lenguaje de programación universal y es el más utilizado y recomendado en planes de estudio de Universidades en todo el mundo y esta cátedra pretende generar en las mentes de los estudiantes el pensamiento estructurado a fin de programar una computadora diseñando algoritmos para resolver problemas y codificarlos en lenguaje C++, depurando cada programa una y otra vez hasta entender la gramática y sus reglas de sintaxis, así como la lógica del programa y la dinámica de la cátedra se desarrolla con el "Aprender practicando".

La cátedra utiliza el aula virtual de la FIUNA en la dirección www.ing.una.py/moodle/ en donde los alumnos pueden acceder a materiales editados por la cátedra y también es el sitio donde se realizan las comunicaciones oficiales para días y horario para exámenes, entrega de trabajos prácticos, publicación de planillas de asistencia, notas de talleres, parciales y finales.

INDICE

Cap. 1 – Introducción a las computadoras y a los lenguajes de programación 04

Cap. 2 – Fundamentos de programación 17

Cap. 3 – El Lenguaje C++: Elementos básicos 26

Cap. 4 – Operadores y Expresiones 33

Cap. 5 – Estructuras de selección: sentencias if y switch 43

Cap. 6 – Estructuras de control: bucles 46

Cap. 7 – Funciones 54

Cap. 8 – Recursividad 58

Cap. 9 – Arrays 60

Cap. 10 – Algoritmos de ordenación 69

Cap. 11 – Estructuras y uniones 86

Cap. 12 – Punteros 93

Apéndice – Ejercicios 105

Bibliografía 124

Capítulo 1

Introducción a las Computadoras y a los Lenguajes de Programación

1.1 Introducción a las computadoras

Una computadora es una máquina electrónica controlada 100% por el ser humano, es decir, funciona de acuerdo a las instrucciones que el hombre le indica. Al unir más de una instrucción en forma lógica y coherente, se crea un programa. Mediante el uso de éstos, la computadora es capaz de recibir, procesar y almacenar información. En otras palabras, una computadora no es útil si no tiene un programa que le indique lo que tiene que hacer.

El valor de una computadora radica en la velocidad y precisión con la cual ésta ejecuta las instrucciones. La capacidad de una computadora se hace específicamente para el ambiente en donde ésta va a operar. Una microcomputadora personal, para la oficina o el hogar, una computadora notebook para cuando uno va de viajes, o una computadora tipo servidor para controlar grandes cantidades de información. Cada una de estas máquinas se ha diseñado para procesar diferentes tipos de información y por ello, cada una de ellas tendrá un valor de acuerdo a su capacidad.

La computadora es más que una calculadora de alta velocidad, se ha convertido en un auxiliar esencial hoy día. Su uso se está implementando en todos los campos de trabajo. Las computadoras están en todos lados y existen en muchas formas, tamaños y colores. Por ejemplo: una calculadora de bolsillo, el reloj de un microondas, el control remoto de un televisor, los juegos de video y los cajeros automáticos son ejemplos entre otros. En trabajos donde el hombre no se concentra o no pone la atención debida a causa de la naturaleza repetitiva de la tarea, una computadora puede realizar la misma tarea durante toda una semana, 24 horas al día, sin pérdida de velocidad ni precisión.

Las ventajas de las computadoras se pueden resumir en cuatro puntos: rapidez, precisión, economía y confiabilidad.

1.2 Generaciones de las computadoras

Primera generación (1946-1959)

Esta generación se caracteriza por el uso de tubos al vacío para conducir la electricidad. Las computadoras de esta generación eran muy grandes en tamaño y lentas al procesar datos. A causa de la gran cantidad de calor que emitían, se requería que siempre estuvieran en un lugar con mucha ventilación. Una vez que las computadoras de esta generación comenzaban un proceso, el mismo no podía ser interrumpido hasta que la computadora lo terminará por completo. Podían realizar 1.000 instrucciones por segundo. Entre las computadoras pertenecientes a esta generación están: la ENIAC y la UNIVAC, siendo estas las primeras computadoras comerciales.

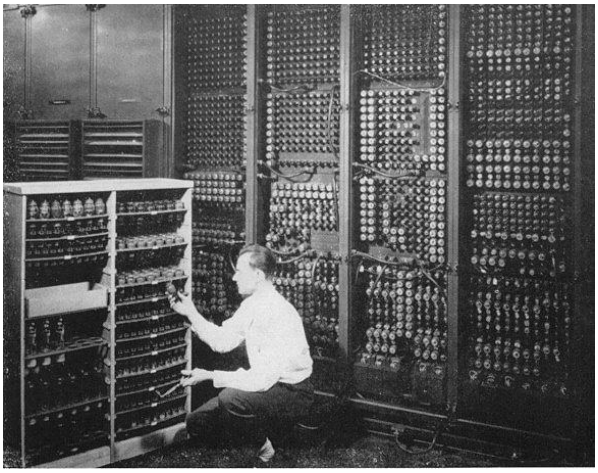


Imagen 1: ENIAC

Segunda generación (1959-1964)

Aparecen los transistores, estos reemplazan los tubos al vacío de la primera generación. Un transistor representa 40 tubos al vacío y son más pequeños y durareros. Las computadoras de esta generación resultaron más económicas ya que consumían menos energía y ocupaban menos espacio. Su capacidad de memoria se amplía al igual que las unidades de entrada y salida de información. Su velocidad de ejecución aumenta y además surgen los primeros lenguajes de computación, ejemplo: FORTRAN. Estas computadoras podían realizar 1.000 instrucciones por segundo.

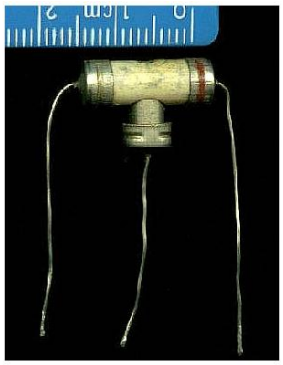


Imagen 2: Transistor

Tercera generación (1965-1971)

Los circuitos integrados pasan a sustituir los transistores. Un circuito integrado (I.C.) es un pequeño encapsulado de sílice que contiene en su interior miles de transistores. Estos proveen mayor velocidad, durabilidad y a su vez son más económicos que los transistores de la segunda generación. Las computadoras de ésa generación son más pequeñas y costosas. Estas computadoras podían realizar 1.000.000 instrucciones por segundo y podían ejecutar varias tareas al mismo tiempo.

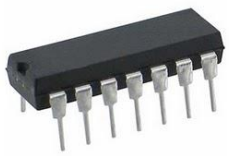


Imagen 3: Circuito integrado

Cuarta generación (1972-)

Los circuitos integrados pasan a integraciones a larga escala, es decir se aumenta la cantidad de transistores de manera considerable en cada circuito integrado. En esta generación aparece el microprocesador. Este a su vez promueve el surgimiento de las microcomputadoras y las computadoras personales, siendo la primera computadora personal la APPLE II en 1977.

El circuito integrado hace que las computadoras de esta generación sean mucho más rápidas. La eficiencia de éstas aumenta considerablemente y se reduce el tamaño y el costo de las mismas.

Imagen 4: Aplle II

1.3 Funciones básicas

Entrada

La unidad de entrada, INPUT, es el término que se utiliza para referirse a la entrada de datos al computador. Entre los medios más usados para la entrada de datos están: el teclado, el mouse.

Proceso

La unidad central de procesamiento (CPU) se encarga de realizar cualquier acción en el interior de la computadora. Es ahí donde todos los procesos y cómputos se llevan a cabo. Comprende la unidad aritmética y lógica, la unidad de Control y la Memoria.

Salida

La unidad de salida, OUTPUT, se utiliza para obtener la información deseada. La información se define como el resultado obtenido luego de procesar los datos. Esta información puede obtenerse mediante papel, monitor, etc.

1.4 Memoria

La memoria es el principal lugar de trabajo de una computadora, ya que todo el procesamiento de información tiene lugar en ella; por ende, la memoria es el recurso más importante de la computadora. Determina tanto la complejidad como el número de programas diferentes que pueden ejecutarse de forma simultánea. La memoria consiste en un archivo electrónico en donde la computadora guarda las instrucciones, programas y/o datos hasta que alguna unidad los solicite.

Existen 3 tipos de memoria:

Random Access Memory (RAM) que es la memoria de acceso aleatoria de almacenamiento temporal. Almacena los datos y programa el proceso, pero su contenido se pierde al apagar el computador. El contenido de esta memoria puede ser modificado.

Read Only Memory (ROM) es la memoria de almacenamiento permanente, sólo puede ser leída. Su contenido no puede ser modificado a menos que se cambien los circuitos dentro de la misma.

Memoria Auxiliar, corresponde al almacenamiento externo del sistema. Esta memoria en realidad está formada por dispositivos tales como: cintas magnéticas y discos flexibles. Su labor es la de complementar la memoria principal.

1.5 Equipos y Programas

Por hardware se entiende, todos los componentes físicos de una computadora (teclado, monitor, impresora, unidad de discos, etc.). El software lo componen aquellos programas que se utilizan en el computador. Un programa se puede definir como una serie o conjunto de instrucciones que le indica a la computadora lo que debe hacer.

Operacionalmente, un sistema de computadoras está formado tanto por el equipo físico como por los programas. El uno no puede trabajar sin el otro. El equipo físico es quien dice que instrucciones pueden ser ejecutadas y los programas le indican al equipo qué hacer mediante el uso de las instrucciones del microprocesador.

Los programas deben escribirse en un lenguaje que la computadora comprenda. Existen una variedad de lenguajes para computadoras. Entre los más importantes están:

Lenguaje de máquina lenguaje básico del computador, consiste de códigos binarios 1 y 0.

Lenguaje de ensamble lenguaje del mismo nivel que el lenguaje máquina, pero que utiliza símbolos en lugar de 1 ó 0.

Lenguaje de alto nivel lenguaje que utiliza términos parecidos al idioma inglés y es usado comúnmente por las personas para resolver un problema. Este es traducido a lenguaje de máquina por medio de un compilador, de manera tal que la computadora lo comprenda.

Aplicaciones parte más cercana al usuario final de la computadora. Es una herramienta construida por lenguajes de programación que ayuda al usuario a laborar día a día en el hogar u oficina.

Las operaciones que debe realizar el hardware son especificadas por una lista de instrucciones, llamadas programas o software. Un programa de software es un conjunto de sentencias o instrucciones al computador. El proceso de escritura o codificación de un programa se denomina programación, y las personas que se especializan en esta actividad se denominan programadores. Existen dos tipos importantes de software: software del sistema y software de aplicaciones. Cada tipo realiza una función diferente.

El software del sistema es un conjunto generalizado de programas que gestiona los recursos del computador, tal como el procesador central, enlaces de comunicaciones y dispositivos periféricos. Los programadores que escriben software del sistema se llaman programadores de sistemas.

El software de aplicaciones es el conjunto de programas escritos por empresas o usuarios individuales o en en equipo y que instruyen a la computadora para que ejecute una tarea específica. Los programadores que escriben software de aplicaciones se llaman programadores de aplicaciones.

1.6 Sistema Operativo

Cuando un usuario interactúa con un computador, la interacción está controlada por el sistema operativo. Un usuario se comunica con un sistema operativo a través de una interfaz de usuario de ese sistema operativo. Los sistemas operativos modernos utilizan una interfaz gráfica de usuario, IGU (Graphical User Interface, GUI) que hae uso masivo de íconos, botones, barras y cuadros de diálogo para realizar tareas que se controlan por el teclado o el mouse entre otros dispositivos. Normalmente el sistema operativo puede residir en un disco que se carga y almacena en la memoria RAM durante el arranque de la computadora en una operación que se denomina arranque del sistema (booting).

El sistema operativo dirige las operaciones globales de la computadora, instruye a la computadora para ejecutar otros programas y controla el almacenamiento y recuperación de archivos (programas y datos) de cintas y discos. Gracias al sistema operativo es posible que el programador pueda introducir y grabar nuevos programas, así como instruir a la computadora para que los ejecute. Los sistemas operativos pueden ser: monousuarios (un solo usuario) y multiusuarios, o tiempo compartido (diferentes usuarios), atendiendo al número de usuario y monocarga (una sola tarea) o multitarea (multiples tareas) según las tareas (procesos) que puede realizar simultáneamente. C++ corre prácticamente en todos los sistemas operativos, Windows 95, Windows NT/2000, Windows XP, Windows 7, UNIX, Linux,… y en casi todas las computadoras personales actuales PC, Mac, Sun, etc.

1.7 Tipos de sistemas operativos

Las diferentes carácterísticas especializadas del sistema operativo permiten a los computadores manejar muchas diferentes tareas así como múltiples usuarios de modo simultáneo o en paralelo, bien de modo secuencial. En base a sus características específicas los sitemas operativos se pueden clasificar en varios grupos:

La multiprogramación permite a múltiples programas compartir recursos de un sistema de computadora en cualquier momento. Con multiprogramación, un grupo de programas se ejecutan alternativamente y se alternan en el uso del procesador. Cuando se utiliza un sistema operativo de un único usuario, la multiprogramación toma el nombre de multitarea.

Un sistema operativo multiusuario es un sistema operativo que tiene la capacidad de permitir que muchos usuarios compartan simultáneamente los recursos de proceso de la computadora. Dada la alta velocidad de transferencia de las operaciones, la sensación es que todos los usuarios están conectados simultáneamente a la CPU.

Un sistema operativo trabaja en multiproceso cuando puede enlazar a dos o mas CPU para trabajar en paralelo en un único sistema de computadora. El sistema operativo puede asignar múltiples CPU para ejecutar diferentes instrucciones del mismo programa o de programas diferentes simultáneamente, dividiendo el trabajo entre las diferentes CPU.

1.8 Lenguajes de programación

La solución de problemas mediante el uso de un computador nos lleva a desarrollar programas o aplicaciones, la construcción de estos programas debe ser realizada mediante una metodología, los pasos para la elaboración de programas son:

1. Diseñar el algoritmo, consiste en analizar el problema y describir los instrucciones de forma ordenada que resuelven el problema.

2. Codificar de forma sistemática la secuencia de instrucciones en un lenguaje.

3. Ejecutar y validar el programa en un computador (verificación)

Para codificar de forma sistemática un algoritmo se necesita de un lenguaje programación que la computadora comprenda.

1.9 Historia de los inicios de la programación:

Charles Babbage, profesor de matemáticas en la universidad de Cambridge en los años de 1828 a 1839. Puede ser considerado como el padre de las computadores, entre 1833 y 1842 Babbage intentó construir una maquina que fuese programable, esta máquina lleva por nombre la maquina Analítica.

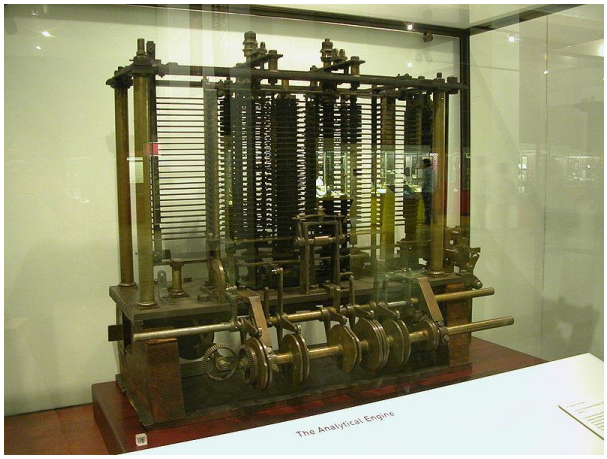


Imagen 5: Máquina analítica de babbage como se puede apreciar en el Science Museum de Londres.

El diseño se basaba en una máquina calculadora, controlada por una secuencia de instrucciones, con una unidad de proceso, una memoria central, facilidades de entrada y salida de datos, y posibilidades de control paso a paso, la secuencia de instrucciones usaba tarjetas perforadas

Ada Augusta Byron (Lady Ada Lovelace), Condesa de Lovelace, matemática; colaboro con Babbage económicamente y promovió activamente la maquina analítica de Babbage.

Lady Ada Lovelace escribió programas para la maquina analítica, estas primeras instrucciones hacen de Ada Lovelace la primera programadora de computadoras en el mundo.

1.10 Lenguajes de Alto Nivel

Hoy día los lenguajes de programación de alto nivel son muy numerosos, aunque la práctica demuestra que su uso mayoritario se reduce a:

C, C++, COBOL, FORTRAN, PASCAL, Vb.NET, JAVA, C#

Internet consume gran cantidad de recursos en forma de lenguajes de programación tales como Java, HTML, XML, Javascript, PHP, etc.

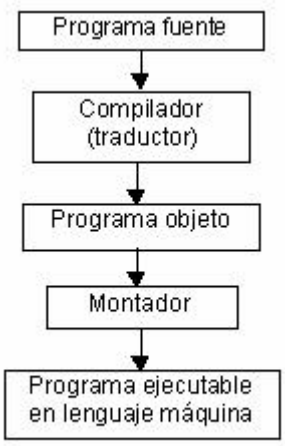
Los traductores de lenguaje son programas que traducen a su vez los programas fuente escritos en lenguajes de alto nivel a código máquina. Los traductores se dividen en compiladores e intérpretes.

Un intérprete es un traductor que toma un programa fuente, lo traduce y a continuación lo ejecuta.

Un compilador es un programa que traduce los programas fuente escritos en lenguaje de alto nivel a lenguaje máquina. Los programas escritos en lenguaje de alto nivel se llaman programas fuente y el programa traducido programa objeto o código objeto. El compilador traduce sentencia a sentencia el programa fuente. Los lenguajes compiladores típicos son C, Pascal, FORTRAN y COBOL.

La compilación y sus fases

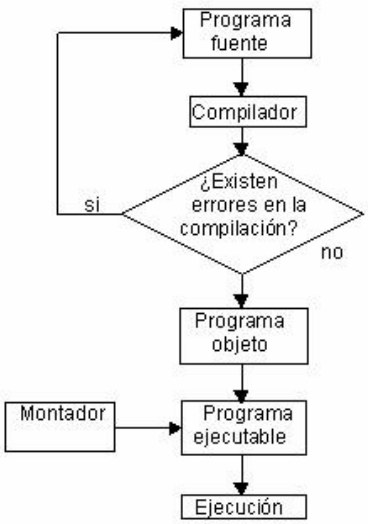
La compilación es el proceso de traducción de programas fuente a programas objeto. El programa objeto obtenido de la compilación ha sido traducido normalmente a código máquina. Para conseguir el programa máquina real se debe utilizar un programa llamdo montador o enlazador (linker). El proceso de montaje conduce a un programa en lenguaje máquina directamente ejecutable.



Fases de la compilación

El proceso de ejecución de un programa escrito en un lenguaje de programación y mediante un compilador suele tener los siguienes pasos:

- Escritura del programa fuente con un editor.
- Compilación del programa utilizando un compilador.
- Verificación y corrección de errores de compilación (listado de errores)
- Obtención del programa objeto
- El montador o enlazador (linker) genera el programa ejecutable
- Se ejecuta el programa y si no existen errores, se obtendrá la salida del programa.



Fases de la ejecución de un programa

1.11 El lenguaje C: historia y características

C es el lenguaje de programación de propósito general asociado, de modo universal, al sistema operativo UNIX. Sin embargo, la popularidad, eficacia y potencia de C, se ha producido porque este lenguaje no está prácticamente asociado a ningún sistema operativo, ni a ninguna máquina en especial. Esta es la razón fundamental por la cual C, es conocido como el lenguaje de programación de sistemas, por excelencia.

C es una evolución de lenguajes BCPL (desarrollado por Martin Richards) y B (desarrollado por Ken Thompson en 1970) para el primitivo UNIX de la computadora DEC PDP-7. C nació realmente en 1978, con la publicación de The C Programming lenguaje, de Brian Kerninghan y Dennis Ritchie (Prentice Hall, 1978). En 1983, el American National Standard Institute (ANSI), que es una organización Americana de estandarización, la cual creó un comité (el denominado X3J11) cuya tarea fundamental consistía en hacer "una definición no ambigua del lenguaje C, e independiente de la máquina". Con esta definición de C se asegura que cualquier fabricante de software que vende un compliador ANSI C incorpora todas las características del lenguaje, especificadas por el estándar. Esto significa también que los programadores que escriban programas en C estándar tendrán la seguridad de que correrán sus modificaciones en cualquier sistema que tenga un compilador C.

C es un lenguaje de alto nivel, que permite programar con instrucciones de lenguaje de propósito general. También C se define como un lenguaje de programación estructurado de propósito general; aunque en su diseño también primó el hecho de que fuera especificado como un lenguaje de programación de sistemas, y ésta característica le proporciona una enorme cantidad de potencia y flexibilidad.

El estándar C formaliza construcciones no propuestas en la primera versión de C, en especial, asignación de estructuras y enumeraciones. Entre otras aportaciones, se definió esencialmente, una nueva forma de declaración de funciones (prototipos). Pero, es esencialmente la biblioteca estándar de funciones, otra de sus grandes aportaciones.

Hoy, en el siglo XXI, C sigue siendo uno de los lenguajes de programación más utilizados en la industria del software, así como en institutos tecnológicos, escuelas de ingeniería y universidades. Prácticamente todos los fabricantes de sistemas operativos, UNIX, Linux, MacOS, Solaris, … soportan diferentes tipos de compiladores de lenguaje C.

El lenguaje C++ se comenzó a desarrollar en 1980. Su autor fue B. Stroustrup, de la ATT. Al comienzo era una extensión del lenguaje C que fue denominada C with classes. Este nuevo lenguaje comenzó a ser utilizado fuera de la ATT en 1983. El nombre C++ fue popularizado también ese año, y hace referencia al carácter del operador incremento de C (++).

Ante la gran difusión y éxito que iba obteniendo en el mundo de los programadores, la ATT comenzó a estandarizarlo internamente en 1987. En 1989 se formó un comité ANSI (seguido algún tiempo después por un comité ISO) para estandarizarlo a nivel americano e internacional.

En la actualidad, el C++ es un lenguaje versátil, potente y general. Su éxito entre los programadores profesionales le ha llevado a ocupar el primer puesto como herramienta de desarrollo de aplicaciones. El C++ mantiene las ventajas del C en cuanto a riqueza de operadores y expresiones, flexibilidad, concisión y eficiencia. Además, ha eliminado algunas de las dificultades y limitaciones del C original. La evolución de C++ ha continuado con la aparición de Java, un lenguaje creado simplificando algunas cosas de C++ y añadiendo otras, que se utiliza para realizar aplicaciones en Internet.

El nombre C++ fue propuesto por Rick Mascitti en el año 1983, cuando el lenguaje fue utilizado por primera vez fuera de un laboratorio científico. Antes se había usado el nombre "C con clases". En C++, la expresión "C++" significa "incremento de C" y se refiere a que C++ es una extensión de C.

Hay que señalar que el C++ ha influido en algunos puntos muy importantes del ANSI C, como por ejemplo en la forma de declarar las funciones, en los punteros a void, etc. En efecto, aunque el C++ es posterior al C, sus primeras versiones son anteriores al ANSI C, y algunas de las mejoras de éste fueron tomadas del C++.

El C++ es a la vez un lenguaje procedural (orientado a algoritmos) y orientado a objetos. Como lenguaje procedural se asemeja al C y es compatible con él. Como lenguaje orientado a objetos se basa en una filosofía completamente diferente, que exige del programador un completo cambio de mentalidad. Las características propias de la Programación Orientada a Objetos (Object Oriented Programming, u OOP) de C++ son modificaciones mayores que cambian radicalmente su naturaleza, en este material no profundizaremos en OOP por ser una cátedra de Introducción a la Programación en Lenguaje C++.

1.12 Ventajas de C

El lenguaje C tiene una gran cantidad de ventajas sobre otros lenguajes, y son, precisamente, la razón fundamental de que después de más de dos décadas de uso, C siga siendo uno de los lenguajes más populares y utilizados en empresas, organizaciones y fábricas de software de todo el mundo. Algunas ventajas que justifican el uso todavía creciente del lenguaje C en la programación de computadoras son:

- El lenguaje C es potente y flexible
- C se utiliza por programadores profesionales para desarrollar software en la mayoría de los modernos sistemas de computadora.
- Se puede utilizar C para desarrollar sistemas operativos, compiladores, sistemas de tiempo real y aplicaciones de comunicaciones.
- Un programa en C puede ser escrito para un tipo de computadora y trasladarse a otra computadora con pocas o ninguna modificación (propiedad conocida como portabilidad)

C se caracteriza por su velocidad de ejecución. En los primeros día de la informática, los problemas de tiempo de ejecución se resolvían escribiendo todo o parte de una aplicación en lenguaje ensamblador (lenguaje muy cercano al lenguaje máquina).

Debido a que existen muchos programas escritos en C, se han creado numerosas bibliotecas C para programadores profesionales que soportan gran variedad de aplicaciones. Existen bibliotecas del lenguaje C que soportan aplicaciones de bases de datos, gráficos, edición de texto, comunicaciones, etc.

En la actualidad son muchos los fabricantes de compiladores C, y se pueden encontrar en el comercio y de distribución gratuita tanto en empresas de distribución como en Internet para los sistemas operativos Windows, Linux, Unix y Mac., entre otros.

Todos los compiladores del lenguaje C++ pueden ejecutar programas escritos en lenguaje C, preferentemente si cumplen el standar ANSI C.

Capítulo 2

Fundamentos de programación

Un programa de computadora es un conjunto de instrucciones (órdenes dadas a la máquina) que producirán la ejecución de una determinada tarea. En esencia, un programa es un medio para conseguir un fin. El fin será normalmente definido como la información necesaria para solucionar un problema.

El proceso de programación es, por consiguiente, un proceso de solución de problemas (como ya se vio anteriormente) y el desarrollo de un programa requiere las siguientes fases:

1. Definición y análisis del problema.
2. Diseño de algoritmos.

   - diagrama de flujo;

   - pseudocódigo.
3. Codificación del programa.
4. Depuración y verificación del programa.
5. Documentación.
6. Mantenimiento.

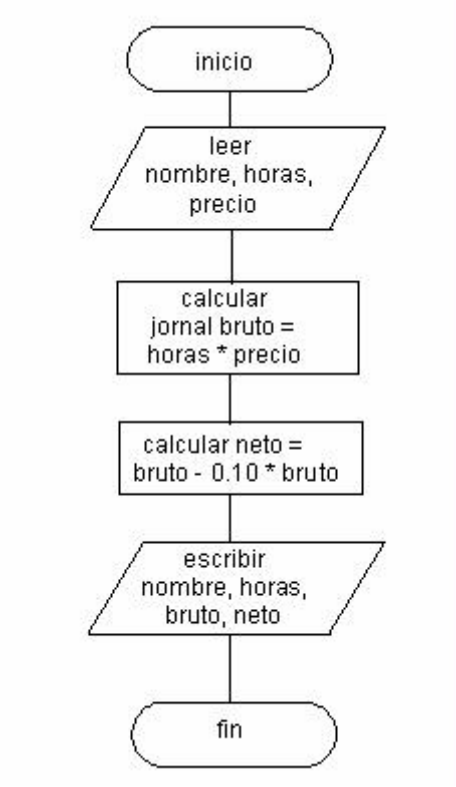
2.1 Diagrama de flujo (*Representación gráfica*)

Un diagrama de flujo (flowchart) es una de las técnicas de representación de algoritmos más antigua y a la vez más utilizada, aunque su empleo ha disminuido considerablemente, sobre todo desde la aparición de lenguajes de programación estructurados. Un diagrama de flujo es un diagrama que utiliza los símbolos (cajas) y que tiene los pasos del algoritmo escritos en esas cajas unidas por flechas, denominadas líneas de flujo, que indican la secuencia en que se deben ejecutar.

Los símbolos estándar normalizados por ANSI (abreviatura de American National Standards Institute) son muy variados, aquí se presentan algunos:

Símbolos principales	Función
	Terminal (Representa el "inicio" y el "fin" de un programa. Puede representar también una interrupción que sea necesario realizar en un programa).
	Entrada/Salida (Cualquier tipo de introducción de datos en la memoria desde los periféricos, "entrada", o registro de la información procesada en un periférico, "salida").
	Proceso (Cualquier tipo de operación que pueda originar cambio de valor, formato o posición de la información almacenada en memoria, operaciones aritméticas, de transferencia, etc.).
no / si	Decisión (Indica operaciones lógicas o de comparación entre datos (normalmente dos) y en función del resultado de la misma determina cuál de los distintos caminos alternativos del programa se debe seguir, generalmente tiene dos salidas(respuestas Si o No), pero puede tener tres o más según los casos).
salidas múltiples	Decisión múltiple (En función del resultado de la comparación se seguirá uno de los diferentes caminos de acuerdo con dicho resultado).
	Conector (Sirve para enlazar dos partes cualesquiera de un ordinograma a través de un conector en la salida y otro conector en la entrada. Se refiere a la conexión en la misma página del diagrama).
← →	Indicador de dirección o línea de flujo (Indica el sentido de ejecución de las operaciones).
	Línea conectora (Sirve de unión entre dos símbolos).
	Conector (Conexion entre dos puntos del organigrama situado en páginas diferentes).
	Llamada a subrutina o a un proceso predeterminado (Una subrutina es un modulo independiente del programa principal, recibe una entrada procedente de dicho programa, realiza una tarea determinada y al terminar regresa al programa principal).

Ejemplo básico de un diagrama de flujo:



El diagrama citado representa la resolución de un programa que deduce el salario neto de un trabajador a partir de la lectura del nombre, horas trabajadas, precio de la hora, y sabiendo que los impuestos aplicados son el 10 % sobre el salario bruto.

2.2 Pseudocódigo

El pseudocódigo es un lenguaje de especificación de algoritmos.

En sí es una mezcla de lenguaje de programación y de lenguaje natural. La idea del pseudocódigo consiste en aprovechar la flexibilidad y poder expresivo del lenguaje natural por un lado, y las reglas de composición de los lenguajes de programación de alto nivel por el otro.

El pseudocódigo utiliza para representar las acciones sucesivas palabras - similares a sus homónimas en los lenguajes de programación -, tales como **Inicio, fin, parar, si-entonces-de lo contrario, mientras** etc. La escritura del pseudocódigo exige normalmente la indentación (sangría en el margen izquierdo) de diferentes líneas. La representación en pseudocódigo del diagrama de flujo del ejemplo anterior sería:

Inicio

{cálculo de impuesto y salario}

leer nombre, hora, precio_hora

salario_bruto ß horas * precio_hora

tasa ß 0.1 *salario_bruto

salario_neto ß salario_bruto - tasa

imprimir nombre, salario_bruto, tasa, salario_neto

fin

El algoritmo comienza con la palabra **Inicio** y finaliza con la palabra **fin**. Entre estas palabras, sólo se escribe una instrucción o acción por línea. La línea encerrada entre llaves { ... } se denomina comentario. Es una información al lector del programa y no realiza ninguna instrucción ejecutable; sólo tiene efecto de documentación interna del programa. Algunos autores suelen utilizar corchetes en lugar de llaves [...].

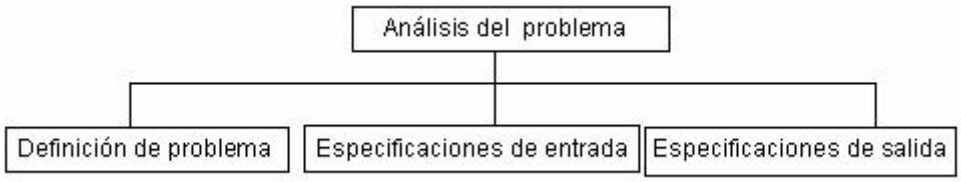
2.3 Metodología de solución

La principal razón para que las personas aprendan a programar en general y los lenguajes de programación en particular es utilizar la computadora como una herramienta para la resolución de problemas ayudado por una computadora. La resolución de un problema consta de ocho etapas:

1. Definición y delimitación del problema a solucionar (enunciado del problema)
2. Pseudocódigo o diagrama de flujo (algoritmo)
3. Prueba de escritorio
4. Codificación
5. Digitación
6. Compilación o interpretación del programa
7. Ejecución del Programa
8. Evaluación de los resultados

2.3.1. - Definición y delimitación del problema a solucionar

El problema debe estar bien definido si se desea llegar a una solución satisfactoria para poder definir con precisión el problema se requiere que las especificaciones de entrada y salida sean descritas con detalle. Una buena definición del problema, junto con una descripción detallada de las especificaciones de entrada y salida, son los requisitos más importantes para llegar a una solución eficaz.



El análisis del problema exige una lectura previa del problema a fin de obtener una idea general de lo que se solicita. La segunda lectura deberá servir para responder a las preguntas:

- ¿Qué información debe proporcionar la resolución del problema?
- ¿Qué datos se necesitan para resolver el problema?

La respuesta a la primera pregunta indicará los resultados deseados o las salidas del problema. La respuesta a la segunda indicará qué datos se proporcionan o las entradas del problema.

2.3.2. - Pseudocódigo o diagrama de flujo (algoritmo)

Una computadora no tiene la capacidad para solucionar problemas más que cuando se le proporcionan los sucesivos pasos a realizar. Estos pasos sucesivos que indican las instrucciones a ejecutar por la máquina constituyen, como ya conocemos, el algoritmo.

En esta etapa es donde se determinan los pasos o instrucciones que deben llevarse a cabo y el orden lógico de su ejecución para dar una eficiente solución al problema.

La información proporcionada al algoritmo constituye su entrada y la información producida por el algoritmo constituye su salida.

2.3.3. - Prueba de escritorio (prueba de un algoritmo)

Para comprobar que un algoritmo realiza la tarea para la cual fue diseñado, debe ejecutarse a mano. Para esto deben utilizarse datos representativos y anotarse los valores que toman las variables en cada paso. Esto se conoce como **prueba de escritorio** (se realiza una prueba para varios valores de las variables de entrada, se verifica si cumple con los resultados esperados para todas las situaciones).

2.3.4. - Codificación

El programa que implementa el algoritmo debe ser escrito en un lenguaje de programación y siguiendo las reglas gramaticales o sintaxis del mismo. La fase de conversión del algoritmo en un lenguaje de programación se denomina codificación, ya que el algoritmo escrito en un lenguaje especifico de programación (lenguaje de alto nivel) se denomina código.

2.3.5. - Almacenamiento

Tras la codificación del programa las instrucciones se graban en un medio legible para la computadora; a igual procedimiento se someten los datos (en disquetes, cassettes, cintas, pendrive, etc.).

2.3.6. - Compilación o interpretación del programa

En esta etapa la computadora chequea si todas las instrucciones están escritas correctamente desde el punto de vista de la sintaxis y gramática de cada lenguaje y las transcribe, dentro de la memoria, del lenguaje de alto nivel al lenguaje máquina para obtener el llamado programa objeto.

2.3.7. - Ejecución del Programa

El programa objeto es ejecutado por la computadora para llegar a los resultados esperados, utilizando los dispositivos, unidades y memoria necesaria, según cada caso o programa.

2.3.8. - Evaluación de los resultados

Obtenidos los resultados se les evalúa para verificar que sean correctos. En caso contrario, se revisa en las etapas anteriores para detectar la falla o error, entrar a corregirla y reiniciar desde este punto los pasos para resolver de nuevo y en forma correcta el problema.

2.4 Prueba de un algoritmo (Ejemplos)

Ejemplo No. 1

Diseñe un algoritmo que dadas 4 calificaciones (Cal1, Cal2, Cal3, Cal4), calcule la calificación promedio y escriba el resultado final junto con un mensaje explicativo.

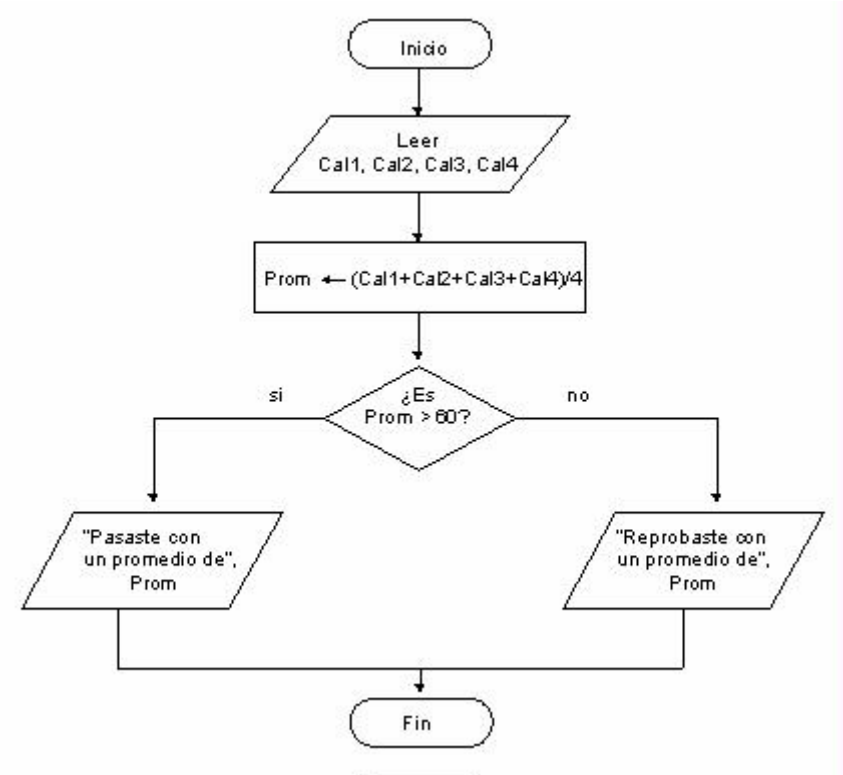
Análisis del problema

Entrada: Leer las calificaciones individuales

Proceso: Calcular la calificación promedio

Salida: Escribir resultado con mensaje explicativo

Diagrama de flujo:



Pseudocódigo:

Inicio

{Cálculo del promedio de calificaciones}

Leer Cal1, Cal2, Cal3, Cal4

Prom = (Cal1+Cal2+Cal3+Cal4)/4

Imprimir ("Resultado final es", Prom)

Fin

Ejemplo No. 2

Escribir un algoritmo para calcular el área de un triángulo dada la base y la altura.

Solución:

Análisis:

La fórmula geométrica del área o superficie de un triángulo es:

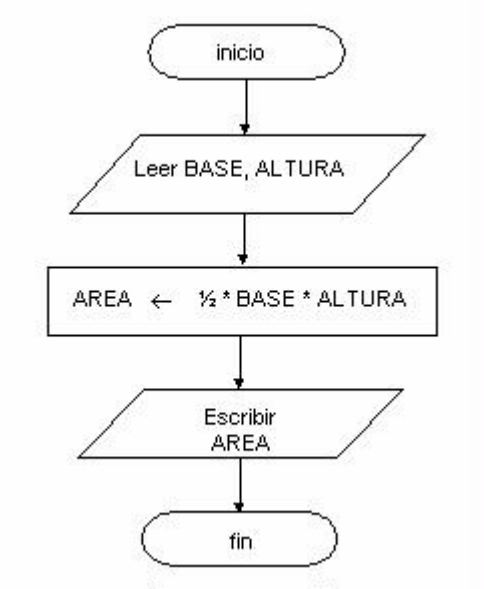
A = (1/2)B*H B = base H = altura

variables: Base, Altura

Suponga que B = 4.5 H = 7.2

A = (1/2) 4.5 * 7.2 = 16.2

Diagrama de flujo:



Capítulo 3

El lenguaje C++: elementos básicos

Una función en C++ es un grupo de instrucciones que realizan una o más acciones. Un programa C++ puede incluir: directivas de preprocesador; declaraciones globales; la función main(); funciones definidas por el usuario; comentarios del programa.

3.1 Directivas del preprocesador

Los compiladores de C++ proporcionan bibliotecas de funciones. Cada biblioteca de funciones tiene asociada un archivo de definición que se denomina ***cabecera***.

Para utilizar algo de una biblioteca en un programa, hay que colocar al principio del programa una ***directiva de preprocesamiento*** seguida de la cabecera de la biblioteca entre ángulos.

Estas directivas constituyen instrucciones al compilador y se escriben iniciando la línea de código con el signo de libro o "almohadilla" (#) y no terminan en punto y coma ya que no son sentencias del lenguaje C++.

La directiva #include indica al compilador que lea el archivo fuente (archivo cabecera o de inclusión) que viene a continuación de ella y su contenido lo inserte en la posición donde se encuentra dicha directiva. Estas instrucciones son de la forma `#include <nombre_archivo.h>`.

La directiva define indica al preprocesador que defina un ítem de datos u operación para el programa C. Por ejemplo, la directiva `#define TAM 10` sustituirá el valor 10 cada vez que TAM aparezca en el programa.

3.2 Declaraciones Globales

Las declaraciones globales indican al usuario que las constantes o variables así declaradas son comunes a todas las funciones de su programa. Se sitúan antes de la función `main()` que es la función principal. La zona de declaraciones globales puede incluir declaraciones de variables además de declaraciones de prototipos de función.

3.3 Función main()

Cada programa C++ contiene una función `main()` que es un punto inicial de entrada al programa. Su estructura es en dos casos usuales:

```
main(){                          int main() {
     Bloque de sentencias;            Bloque de sentencias;
}                                return 0; }
```

Además de la función `main()`, un programa C++ consta de una colección de subprogramas que en C++ siempre son funciones. Las sentencias de C++ situadas en el cuerpo de al función `main()`, o de cualquier otra función, deben terminar en punto y coma.

3.4 Elementos de un programa C++

Un programa en C++ es una secuencia de caracteres que se agrupan en componentes léxicos (tokens) que comprenden el vocabulario básico del lenguaje. Estos componentes de léxico son: palabras reservadas, identificadores, constantes, constantes de cadena, operadores y signos de puntuación.

3.5 Caracteres

Los caracteres que se pueden utilizar para construir elementos del lenguaje (componentes léxicos o tokens) son:

abcdefghi jklmnopqrstuvwxyz
ABCDEFCHIJKLMNOPQRSTUVWXYZ
0123456789
caracteres espacio (blancos y tabulaciones)

3.6 Comentarios

C++ soporta dos tipos de comentarios. Las líneas de comentarios al estilo C y C ANSI, tal como:

/* Comentario estilo C */

/* Comentario mas extenso, pero también es estilo C y ANSI C */

El otro tipo de comentarios se pueden utilizar por los programadores de C++:

La versión /*...* / se utiliza para comentarios que excedan una línea de longitud y la versión //... se utiliza, sólo, para comentarios de una línea. Los comentarios no se anidan.

3.7 Identificadores

Los identificadores (nombres de variables, constantes,...) deben comenzar con una letra del alfabeto (mayúscula o minúscula) o con un carácter subrayado, y pueden tener uno o más caracteres. Los caracteres segundo y posteriores pueden ser: letras, dígitos o un subrayado, no permitiéndose caracteres no alfanuméricos ni espacios.

```
tescprueba // legal
X123 // legal
multi_palabra // legal
var25 // legal
15var // no legal
```

C++ es sensible a las mayúsculas.

`Paga_mes` es un identificador distinto `a paga_mes`

Una buena práctica de programación aconseja utilizar identificadores significativos que ayudan a documentar un programa.

```
nombre apellidos salario precio_neto
```

3.8 Palabras reservadas

Las palabras reservadas o claves no se deben utilizar como identificadores, debido a su significado estricto en C++; tampoco se deben redefinir. La Tabla B. 1 enumera las palabras reservadas de C++ según el ARM(Siglas del libro de BJARNE STROUSTRUP en el que se definen las reglas de sintaxis del lenguaje C++ estándar).

Tabla B.1. Palabras reservadas de C++

asm*	continue	float	new*	signed	try
auto	default	for	operator*	sizeof	typedef
break	delete*	friend*	private*	static	union
case	do	goto	protected*	struct	unsigned
catch*	double	if	public*	switch	virtual*
char	else	inline*	register	template*	void
class*	enurn	int	return	this*	volatile
const	extern	long	short	throw*	while

*Estas palabras no existen en ANSI C.

Los diferentes compiladores comerciales de C++ pueden incluir, además, nuevas palabras reservadas.

3.9 Tipos de datos

Los tipos de datos simples en C++ se dividen en dos grandes grupos: integrales (datos enteros) y de coma flotante (datos reales). La Tabla B.5. muestra los diferentes tipos de datos en C++,

Tipos de datos simples en C++

char	signed	char
unsigned	char	short
int	long	unsigned
short	unsigned	unsigned long
float	double	long double

Los tipos derivados en C++ pueden ser:

- enumeraciones (enum),
- estructuras (struct),
- uniones (union),
- arrays,
- clases (class y struct),
- uniones y enumeraciones anónimas,
- punteros,

3.11 Constantes

C++ contiene constantes para cada tipo de dato simple (integer, char,...). Las constantes pueden tener dos sufijos, u, l y f. Que indican tipos unsigned, long y float, respectivamente. Así mismo, se pueden añadir los prefijos o y ox, que representan constantes octales y hexadecimales.

```
456 0456 0x476 // constante enteras : decima1, octal ,
hexadecimal
1231 123ul // constante enteras :long, unsigned, long
´B´ ´b´ ´4´ // constante de tipo char
3.1415f // constante reales de diferente posición
"cadena de caracteres" // Constante de cadena
```

Las cadenas de caracteres se encierran entre comillas dobles (“), y las constantes de un solo carácter se encierran entre comillas simples (‘).

3.12 Declaración de constantes

En C++, los identificadores de variables/constantes se pueden declarar constantes, significando que su valor no se puede modificar.

Esta declaración se realiza con la palabra reservada const.

```
const double PI= 3.11416;
const char BLANCO = ´ ´;
const double PI_EG = -I;
const double DOBLE_I = 2 * PI ;
```

3.13 Declaración de variables

En ANSI C, todas las declaraciones de variables y funciones se deben hacer al principio del programa o función. Si se necesitan declaraciones adicionales, el programador debe volver al bloque de declaraciones al objeto de hacer los ajustes o inserciones necesarios. Todas las declaraciones deben hacerse antes de que se ejecute cualquier sentencia. Así, la declaración típica en C++,

```
NombreTipo Nombrevariable1, Nombrevariable2;
```

proporciona declaraciones tales como:

```
int saldo, meses;
double clipper, salario;
```

Al igual que en C, se pueden asignar valores a las variables en C++:

```
int mes =4, dia, anio=1995;
doble salario = 45.675;
```

En C++, las declaraciones de variables se pueden situar en cualquier parte de un programa. Esta característica hace que el programador declare sus variables en la proximidad del lugar donde se utilizan las sentencias de su programa.

3.14 Conversión de tipos

La conversiones explícitas se fuerzan mediante moldes (casts). La conversión forzosa de tipos de C tiene el formato clásico:

```
(tipo) expresión
```

C++ ha modificado la notación anterior por una notación funcional como alternativa sintáctica:

```
nombre del tipo (expresion)
```

Las notaciones siguientes son equivalentes:

```
 float(x); //notacion de casteo en C++

 (float)x; //notacion de casteo en C
```

```
Ejemplo:

main(){
      float x=6,58;
      y=int(x);
      cout<<x<<y;
}
```

Capítulo 4

Operadores y Expresiones

4.1 Operadores

C++ es un lenguaje muy rico en operadores. Se clasifican en los siguientes grupos:

- Aritméticos.
- Relacionales y lógicos.
- Asignación.
- Acceso a datos y tamaño.
- Manipulación de bits.
- Varios.

Como consecuencia de la gran cantidad de operadores, se producen también una gran cantidad de expresiones diferentes.

4.1.1 Operadores aritméticos

C++ proporciona diferentes operadores que relacionan operaciones aritméticas.

Operador	Nombre	Propósito	Ejemplo
+	Más unitario	Valor positivo de x	x = +y,+5
-	Negación	Valor negativo de x	x = -y;
+	Suma	Suma x e y	z = x + y;
-	Resta	Resta y de x	z = x - y;
*	Multiplicación	Multiplica x por y	y = x * y;
/	División	Divide x por y	z = x / y;
%	Módulo	Resto de division	z= x % y;
++		Incremento	Incrementa x después de usar x++
--		Decremento	Decrementa x antes de usar --x

Ejemplo:

```
-i                  // menos unitario
+w;                 // mas unitario
a * b               // multiplicación
b / c               // división
a % t;              // modulo
a + b               // suma
a - b               // resta
int a = 5/2;        // a toma el valor 2, porque a es entero
float b = 5/2;      // b toma el valor 2.5, porque b es real
```

Los operadores de incremento y decremento sirven para incrementar y decrementar en una unidad los valores almacenados en una variable.

```
variable ++        // postincremento
++variable         // preincremento
variable --        // postdecremento
--variable         // predecremento
++a o a++;  equivale a  a = a + 1;
--b o b--;  equivale a  b = b – 1;
```

Los formatos postfijos se conforman de modo diferente según la expresión en que se aplica

```
b = ++a ;   equivale a  a = a + 1; b = a;
b = a--;    equivale a  b = a;   a = a - 1;

int i, j, k = 5;
k++;               // k vale 6, igual efecto que ++k
--k;               // k vale ahora 5, igual efecto que k- –
i = 4 * k++;       // k es ahora 6 e i es 20
k = 5;
j = 4* ++k;        // k es ahora 6 e i es 24
```

4.1.2 Operadores de asignación

El operador de asignación (=) hace que el valor situado a la derecha del operador se adjudique variable situada a su izquierda. La asignación suele ocurrir como parte de una expresión de asignación y las conversiones se producen implícitamente. La asignación es solo de derecha a izquierda.

```
z = b + 5; //asigne (b + 5) a variable z
```

C++ permite asignaciones múltiples en una sola sentencia. Así:

`a = b + (c=10);` equivale a: `c = 10; a = b + c;`

otros ejemplos de expresiones válidas y no válidas son:

```
// expresiones legales          //expresiones no legales
a = 5 * (b+a);                  a+3=b;
doble x = y;                    PI = 3; // siendo PI una constante
a = b = c;                      x++ = y;
```

C++ proporciona operadores de asignación que combinan operadores de asignación y otros diferentes, produciendo operadores tales como +=, /=, -=, *= y %= . C++ soporta otros de operadores de asignación para manipulación de bits.

Operadores aritméticos de asignación:

```
+=    x=x+y;      x+=y;
-=    x=x-y;      x-=y;
=     x=x* y;     x*=y;
/=    x=x/y;      x/=y;
%=    x=x%y;      X%=Y;
```

Ejemplos:

```
a + b;          equivale    a     a =   a + b;
a * = a+b;      equivale    a     a =   a * (a + b);
v + = e;        equivale    a     v =   v + e;
v − = e,        equivale    a     v =   v - e;
```

Expresiones equivalentes

```
n = n+1
n+=1;
n++;
++n;
```

Operadores lógicos y relacionales

Los operadores lógicos y relacionales son los bloques de construcción básicos para constructores de toma de decisión en un lenguaje de programación. A continuación se muestran los operadores lógicos y relacionales:

```
Operador    Descripción            Ejemplo
&&          AND (y) lógico         a && b
||          OR (o) lógico          c || b
!           NOT (no) lógico        !c
<           Menor que              i<c
<=          Menor o igual que      i<=0
>           Mayor que              j>50
>=          Mayor o igual que      j>=8.5
==          Igual a                x=='\0'
!=          No igual a             c!='\n'
?:          Asignación condicional k= (i < 5) ? i=1 ;
```

El operador ?: se conoce como expresión condicional, La expresión condicional es una abreviatura de la sentencia condicional if_else.

La sentencia if

```
if(condicion)
            variable = exprexion1;
      else
            variable = exprexion2;
es equivalente a
variable=(condicion) ? expresion1:expresion2;
```

La expresión condicional comprueba la condición. Si esa condición es verdadera, se asigna expresionl a variable; en caso contrario, se asigna expresion2 a variable.

Reglas prácticas

Los operadores lógicos y relacionales actúan sobre valores lógicos el valor/es puede ser o bien 0, o bien el puntero nulo, o bien 0.0; el valor verdadero puede ser cualquier valor distinto de cero.

La siguiente tabla muestra los resultados de diferentes expresiones.

```
x >  y      distinto a 0 si x excede a y, sino 0
x >= y      distinto a 0 si x es mayor o igual a y,sino 0
x <  y      distinto a 0 si x es menor que y, sino 0
x <= y      distinto a 0 si x es menor que o igual a y, sino 0
x == y      distinto a 0 si x es igual a y, sino 0
x != y      distinto a 0 si x e y son distintos, sino 0
!x          distinto a 0 si x es 0,sino 0
x || y      0 si ambos x e y son 0, sino distinto a 0
```

Evaluación en cortocircuito

C++, igual que C, admite reducir el tiempo de las operaciones lógicas; la evaluación de las expresiones se reduce cuando alguno de los operandos toman valores concretos.

1. Operación lógica AND (&&) Si en la expresión expr1 && expr2, expr1 toma el valor cero y la operación lógica AND (y) siempre será cero, sea cual sea el valor de expr2. En consecuencia, expx2 no se evaluará nunca.
2. Operación lógica OR (II). Si expr1 toma un valor distinto de cero, la expresión expr1 || expr2 se evaluará a 1, cualquiera que sea el valor de expr2; en consecuencia, expr2 no se evaluará.

Operaciones de manipulación de bits

C++ proporciona operadores de manipulación de bits, así como operadores de asignación de manipulación de bits.

Operadores de manipulación de bits (bitwise):

Operador	Descripción	Ejemplo
&	AND bit a bit	x & 128
|	OR bit a bit	j | 64
^	XOR bitabit	j ^ 2
~	NOT bit a bit	~ j
<<	Desplazar a la izquierda (mult. por 6)	i << 3
>>	Desplazar a la derecha (div. por 8)	j >> 4

Operadores de asignación de manipulación de bits:

Operador	Formato largo	Formato corto
&=	x = x & y	x& = y;
|=	x = x | y	x | = y;
^=	x = x ^ y	x ^= y ;
<<=	x = x << y	x << = y ;
>>=	x = x >> y	x >> = y ;

Ejemplo:

~x	Cambia *los bits 1* a 0 y *los bits 0 a 1*
y & y	*Operación lógica AND (y) bit a bit de x e y*
x | y	*Operación lógica GR (o) bit a bit de x e y*
x << x	*se desplaza a la izquierda (en y posiciones)*
x >> x	*se desplaza a* la *derecha (en y posiciones)*

El Operador sizeof

El operador sizeof proporciona el tamaño en bytes de un tipo de dato o variable sizeof toma el argumento correspondiente (tipo escalar, array, record, etc.). La sintaxis del operador es: sizeof (nombre variable | tipo de dato)

Ejemplo:

```
int m , n[12]
sizeof(m)    // proporciona 4, en maquinas de 32 bits
sizeof(n)    // proporciona 48
sizeof (15)  // proporciona 4
sizeof (int) // da el tamaño en memoria de un int (generalmente 32 bits)
```

Prioridad y asociatividad de operadores

Cuando se realizan expresiones en las que se mezclan operadores diferentes es preciso establecer una precedencia (prioridad) de los operadores y la dirección (o secuencia) de evaluación (orden de evaluación: izquierda-derecha, derecha-izquierda), denominada asociatividad.

Precedencia y asociatividad de operadores.

Ejemplo:

a * b/c+d *equivale* a (a * b) / (c + d)

:: ( ) [ ] . ->	Izquierda-Derecha
++ --	Derecha-Izquierda
& *(dirección)* ~ *(tipo)* ¡ - +	
sizeof *(tipo)* new delete * *(indireccion)*	
.* ->*	Izquierda-Derecha
* / %	Izquierda-Derecha
+ —	Izquierda-Derecha
	Izquierda-Derecha
	Izquierda-Derecha
<< >>	Izquierda-Derecha
< <= > >=	Izquierda-Derecha
== ¡=	Izquierda-Derecha
&	Izquierda-Derecha
^	Izquierda-Derecha
|	Izquierda-Derecha
&&	Izquierda-Derecha
||	Izquierda-Derecha
= += -= *= /= %= >>= <<= &= |= ^=	Derecha-Izquierda
, *(operador coma)*	Izquierda-Derecha

Sobrecarga de operadores

La mayoría de los operadores de C++ pueden ser sobrecargados o redefinidos para trabajar con nuevos tipos de datos

Operadores que se pueden sobrecargar:

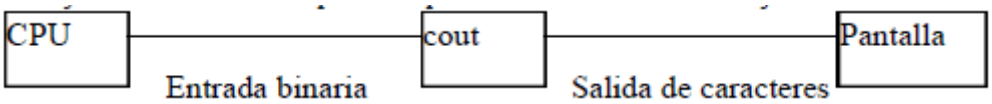
+ - * / % ^ & | ~ ! = < > >> << >>= <<= != == () -> , []

Entradas y salidas básicas

Al contrario que muchos lenguajes, C++ no tiene facilidades incorporadas para manejar entrada o salida. En su lugar, se manejan por rutinas de bibliotecas. Las clases que C++ utiliza para entrada y salida se conocen como flujos. Un flujo (stream) es una secuencia de caracteres junto con una colección de rutinas para insertar caracteres en flujos (a pantalla) y extraer caracteres de un flujo (de teclado).

Salida

El flujo cout es el flujo de salida estándar que corresponde a stdout en C. Este flujo se deriva de la clase ostream construida en iostream.



Si se desea visualizar el valor del objeto int llamado i, se escribe la sentencia:

```
cout << i;
```

El siguiente programa visualiza en pantalla una frase:

```
#include <iostream>
using namespace std;
main()
{
      cout << "Hola Mundo\n";
}
```

Las salidas en C++ se pueden conectar en cascada, con una facilidad de escritura mayor que en C.

```
#include <iostream>
using namespace std;
main()
{
      int i;
      i = 1099;
      cout << ”El valor de i es” << i << “\n”;
}
```

Otro programa que muestra la conexión en cascada es:

```
#include <iostream>
using namespace std;
main()
{
      int x = 45;
      double y = 496.125;
      char *c = "y es multiplicada por x=";
      cout << c << y * x << "\n”;
}
```

Entrada

La entrada se maneja por la clase istream. Existe un objeto predefinido istream, llamado cin, que se refiere al dispositivo de entrada estándar (el teclado).

El operador que se utiliza para obtener un valor del teclado es el operador de extracción >>. Por ejemplo, si i era un objeto int, se escribirá:

```
cin >> i;
```

que obtiene un número del teclado y lo almacena en la variable i.

Un programa simple que lee un dato entero y lo visualiza en pantalla es:

```
#include <iostream>
using namespace std;
main()
{
      int i;
      cin >> i;
      cout << i;
}
```

Al igual que en el caso de cout, se pueden introducir datos en cascada:

```
#include <iostream>
using namespace std;
main()
{
      char c[60];
      int x,y;
      cin >> c >> x >> y;
      cout << c << ” ” << x << ” ” << y << “\n”;
}
```

Manipuladores

Un método fácil de cambiar la anchura del flujo y otras variables de formato es utilizar un operador especial denominado manipulador. Un manipulador acepta una referencia de flujo como un argumento y devuelve una referencia al mismo flujo.
El siguiente programa muestra el uso de manipuladores específicamente para conversiones de número (dec, oct, y hex):

```
#include <iostream>
using namespace std;
main ()
{
      int i = 36;
      cout << dec << i << oct << i << " " << hex << i <<
      "\n";
}
```

La salida de este programa es: 36 44 24

Otro manipulador típico es endl, que representa al carácter de nueva línea (salto de línea), es equivalente a \ n . El programa anterior se puede escribir también así:

```
#include <iostream>
using namespace std;
main ()
{
      int i = 36;
      cout << dec << i << oct << i << hex << i << endl;
}
```

Sentencias

Un programa en C++ consta de una secuencia de sentencias. Existen diversos tipos de sentencias. El punto y coma se utiliza como elemento terminal de cada sentencia.

Sentencia de declaración

Se utilizan para establecer la existencia y, opcionalmente, los valores iniciales de objetos identificados por nombre.

nombreTipo identificador, …;
nombreTipo identificador = expresión, …;
const NombreTipo identificador = expresión, …;

Algunas sentencias válidas en C++ son:

```
char cl;
int p, q = 5, r =a + b; //suponiendo que a y b han sido
declaradas e inicializadas antes
const double IVA = 10.0;
```

Sentencias de expresión

Las sentencias de expresiones hacen que la expresión sea evaluada. Su formato general es:

expresión;

Ejemplo:

```
n++;
425; // legal, pero no hace nada
a + b; // legal, pero no hace nada
a < b || b != 0;
a += b = 3; // sentencia compleja
```

C++ permite asignaciones múltiples en una sentencia.

```
m = n + (p=5) equivale a p=5; m = n + p ;
```

Sentencias compuestas

Una sentencia compuesta es una serie de sentencias encerradas entre llaves. Las sentencias compuestas tienen el formato:

```
{
        Sentencia;
        Sentencia;
        Sentencia;
        ....
}
```

Las sentencias encerradas pueden ser cualquiera: declaraciones, expresiones, sentencias compuestas, etc.

Un ejemplo es:

```
int i = 5;
double x = 3.14, y = -4.25;
int j = 4 - i
x = 4.5 * (x - y);
```

El cuerpo de una función C++ es siempre una sentencia compuesta.

Capítulo 5

Estructuras de selección: sentencias if y switch

Los programas definidos hasta este punto se ejecutan de modo secuencial, es decir, una sentencia después de otra. La ejecución comienza con la primera sentencia del programa y prosigue hasta la última sentencia, cada una de las cuales se ejecuta una sola vez. Esta forma de programación sólo permite resolver programas sencillos. Sin embargo, para la resolución de problemas de tipo general se necesita la capacidad de controlar cuáles son las sentencias que se ejecutan, y en qué momentos.

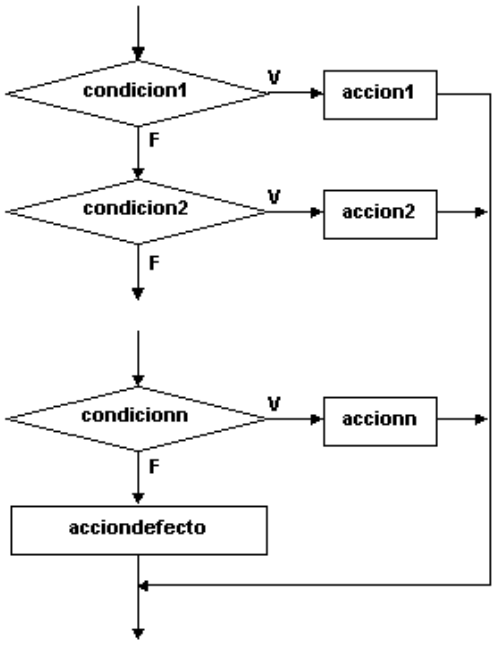
Estructuras de Selección

Las estructuras de selección o condicionales controlan si una sentencia o secuencia de sentencias se ejecutan, en función del cumplimiento o no de una condición o expresión lógica. C++ tiene dos estructuras de control para la selección, if y switch.

Sentencia if

La sentencia if elige entre varias alternativas en base al valor de una o más expresiones lógicas.

```
if (condicion1)
{
      // Sentencias SI
      // Acción 1
}
else if (condicion2)
{
      // Sentencias SINO
      // Acción 2
}
else
{
      //Acción por defecto
}
```



Sentencia switch

La sentencia switch es una sentencia de C++ que se utiliza para seleccionar una de entre múltiples alternativas. Esta sentencia es especialmente útil cuando la selección se basa en el valor de una variable de un tipo simple o de una expresión de un tipo simple denominada expresión de control o selector.

```
switch (numero)
{   case 1:
    case 3:
    case 5: // Sentencias para los valores 1,3 y 5
      break;
    case 2: // Sentencias para el valor 2
      break;
    case 4: // Sentencias para el valor 4
      break;
    default : // Sentencias EN OTRO CASO
      break;
}
```

Ejemplo en el que usan sentencias if y switch:

Escribir un programa que lea un número entero entre 1 y 5. Si el número es menor que 1 o mayor que 5 el programa debe avisar y no hacer nada y en caso de que sea válido debe mostrar el número romano que lo representa.

```
#include <iostream>
using namespace std;
const int MIN = 1;
const int MAX = 5;
int main()
{
   int num;
   cout << "Introduzca un número entre " <<MIN<< " y " <<MAX<< ": ";
   cin >> num;
   if (num<MIN)
   {
      cout << "Error: El Número debe ser mayor que " << MIN << endl;
   }
   else if (num>MAX)
   {
      cout << "Error: El Número debe ser menor que " << MAX << endl;
   }
   Else
{
switch(num)
{
case 1: cout << num << " = I " << endl;
break;
case 2: cout << num << " = II " << endl;
break;
case 3: cout << num << " = III " << endl;
break;
case 4: cout << num << " = IV " << endl;
break;
case 5: cout << num << " = V " << endl;
break;
}
}
system("PAUSE"); // Hace una pausa
return 0; // Valor de retorno a S.O.
}
```

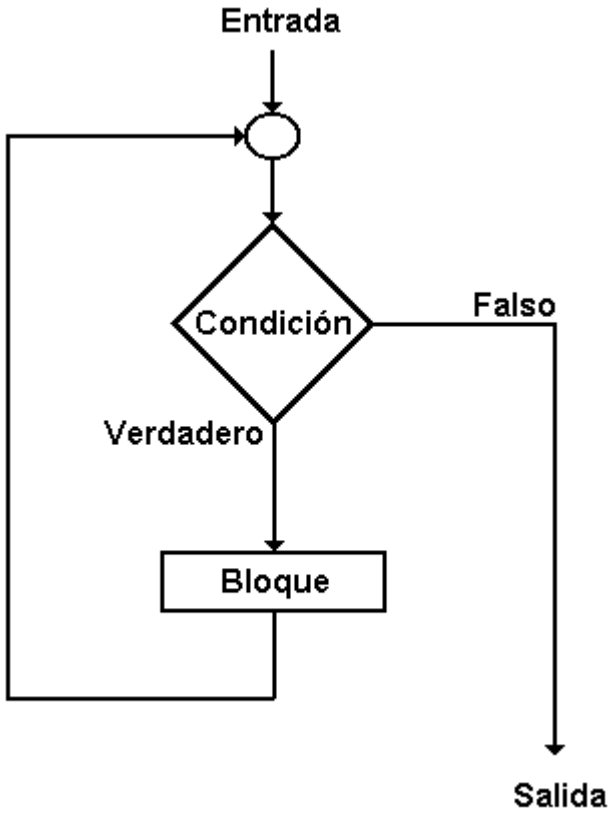
Capítulo 6

Estructuras de control: bucles

C++ soporta tres tipos de estructuras de iteración: los bucles while, do-while y for.

Sentencia while

Un bucle while tiene una condición de control o expresión lógica (<expres_log>), que ha de ir encerrada entre paréntesis, que controla la secuencia de repetición. La posición de esta condición es delante del cuerpo del bucle, y por este motivo el bucle while es un bucle preprueba, ya que se evalúa la condición antes de que se ejecute el cuerpo del bucle. El cuerpo del bucle se ejecuta mientras se cumpla la condición de control. Si la condición es falsa, entonces el cuerpo no se ejecuta.



Hay que hacer notar que si la condición es cierta inicialmente, la sentencia while no terminará nunca (bucle infinito) a menos que en el cuerpo de la misma se modifique de alguna forma la condición de control del bucle. Una sentencia while se ejecutará cero (condición de control inicialmente falsa) o más veces.

La sintaxis de la sentencia es la siguiente:

```
while (<condicion>)
{
   <sentencias>;
}
```

Como en el caso de la sentencia de selección simple o doble, la utilización de llaves ({ }) antes y después del bloque de sentencias únicamente es obligatorio si el número se sentencias del bloque es mayor o igual que dos.

Por ejemplo, si queremos leer una secuencia de enteros y calcular su suma deteniéndonos cuando se lea un número negativo, podemos hacer el siguiente programa:

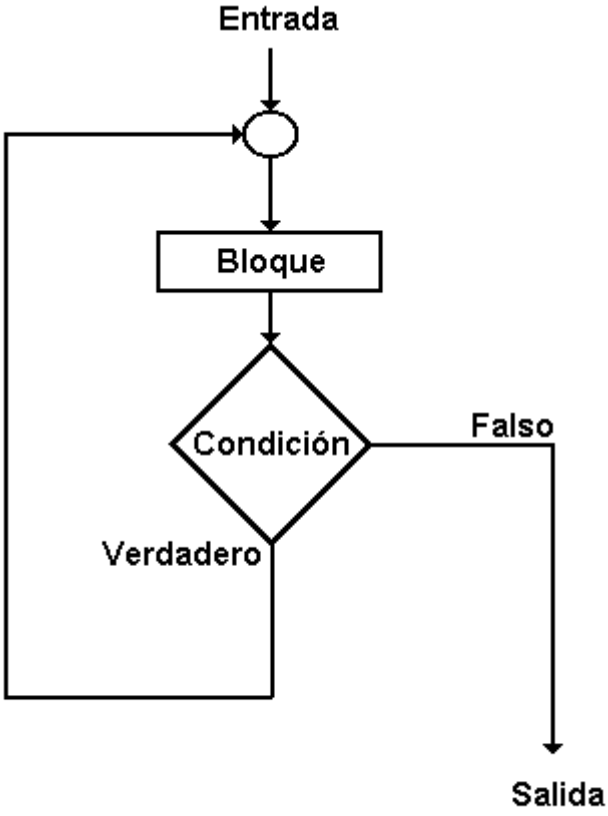
```
#include <iostream>
using namespace std;
int main()
{
   int sum, i;
   cout << "Introduzca un numero:";
   cin >> i; //lectura adelantada
   sum = 0;
   while (i >= 0)
   {
      sum = sum + i;
      cin >> i; //actualización
   }
   cout << "El resultado es:";
   cout << sum << endl;
}
```

Este otro ejemplo visualiza por pantalla n asteriscos:

```
#include <iostream>
using namespace std;
main()
{
   int n=20;
   int contador;
   contador=0; //inicialización
   while (contador < n)
   {
      cout << " * ";
      contador++; //incremento
   }
}
```

Sentencia do-while

En esta sentencia, el cuerpo del bucle se ejecuta hasta que sea falsa la expresión lógica (<expres_log>)(que ha de ir encerrada entre paréntesis). Por tanto, al igual que en el bucle while el cuerpo del bucle se ejecuta mientras la expresión lógica sea cierta. El bucle do-while también se denomina post-prueba, ya que la expresión lógica se comprueba cada vez después de la ejecución del cuerpo del bucle.



La sintaxis del bucle do-while es la siguiente:

```
do
{
   <sentencias>;
}
while (<condicion>);
```

Como en el caso anterior, la utilización de llaves ({ }) antes y después del bloque de sentencias únicamente es obligatorio si el número se sentencias del bloque es mayor o igual que dos. Sin embargo, se aconseja siempre su uso en prevención de posibles errores.

El cuerpo de una sentencia do-while siempre se ejecuta al menos una vez. Cuando esto deba ocurrir en un programa (el cuerpo del bucle tenga que ejecutarse una o más veces), es conveniente el uso de la sentencia do-while en lugar de la sentencia while. Por ejemplo, en lugar de escribir:

```
#include <iostream>
using namespace std;
main()
{
   int numero;
   cout << "Introduzca un numero entre 1 y 10";
   cin >> numero;
   while !((numero>=1) && (numero <= 10))
   {
     cout << "Introduzca un número entre 1 y 10");
     cin >> numero;
   }
}
```

podemos escribir:

```
#include <iostream>
using namespace std;
main()
{
   int numero;
   do
   {
      cout << "Introduzca un numero entre 1 y 10";
      cin >> numero;
   }
   while ((numero < 1) || (numero > 10));
}
```

La primera ejecución del bucle da a numero un valor, de manera que no necesitamos inicializarlo antes del comienzo del bucle.

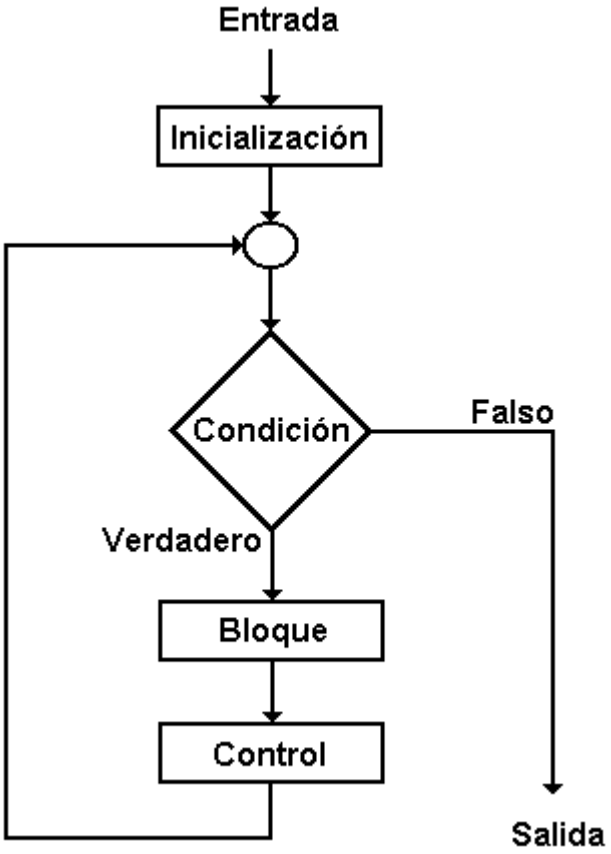
En el ciclo do-while, la línea con la sentencia while termina en punto y coma indefectiblemente.

Sentencia for

Una estructura típica for sería la siguiente:

```
for ( [<inicializacion>]; [<condicion>] ; [<incremento>] )
{   <sentencia>;
}
```

donde (<inicializacion>) es la parte en la que se inicializa la variable de control del bucle, (<condicion>) es una expresión lógica que hace que el bucle ejecute el bloque de sentencias mientras que dicha expresión sea cierta, e (<incremento>) es una expresión que decrementa o incrementa la variable de control del bucle. De esta manera para cada valor que tome la variable de control de bucle, el cuerpo del bucle se ejecuta una vez. Cuando la variable de control del bucle toma su primer valor, (<inicializacion>), ésta se va modificando automáticamente en cada iteración del bucle de acuerdo al valor de incremento (<incremento>) mientras que la expresión lógica (<condicion>) sea falsa. En el caso de que cuando la variable de control del bucle tome su primer valor la expresión lógica sea falsa, el cuerpo del bucle no se ejecutará ninguna vez.



La utilización de llaves ({ }) antes y después del bloque de sentencias únicamente es obligatoria si el número se sentencias del bloque es mayor o igual que dos. Sin embargo, se aconseja siempre su uso en prevención de posibles errores.

Existen dos formas de implementar la sentencia for: (i) formato ascendente, en el que la variable de control se incrementa, y (ii) formato descendente, en el que la variable de control se decrementa. A continuación mostramos dos ejemplos de formato ascendente y descendente:

```
#include <iostream>
using namespace std;
main()
{
   // formato ascendente
   int n;
   for (n = 1; n <= 10; n++)
   {
      cout << n << endl;
   }
}
```

```
#include <iostream>
using namespace std;
main()
{
    // formato descendente
   int i;
   for (i = 9; i >= 0; i -= 3)
   {
      cout << i << endl;
   }
}
```

En el ejemplo de formato ascendente la variable de control es n y su valor inicial es 1, el valor límite es 10 y la expresión de incremento es n++. Esto significa que el bucle ejecuta la sentencia del cuerpo del bucle una vez por cada valor de n en orden ascendente de 1 a 10, por tanto el bucle se ejecutará 10 veces. En la primera iteración n tomará el valor 1, en la segunda el valor 2, y así sucesivamente hasta el valor 10. El ejemplo de formato descendente inicializa la variable de control i a 9. El bucle se ejecuta mientras i no sea negativo, y como la variable de control del bucle se decrementa en 3, el bucle se ejecutará 4 veces con el valor de la variable de control i, 9, 6, 3 y 0.

C++ permite declarar la variable de control de bucle al mismo tiempo que se inicializa, sin embargo nosotros siempre la declararemos en la zona de declaraciones al principio del programa.

La variable de control se puede incrementar o decrementar en valores de tipo int, pero también es posible en valores de tipo double , y por tanto estaríamos incrementando en una cantidad decimal.

La expresión de incremento no necesita ser una suma o una resta. Tampoco se requiere que la inicialización de una variable de control sea igual a una constante. Se puede inicializar y modificar una variable de control de bucle en cualquier cantidad que se desee.

Lógicamente, cuando la variable de control no sea de tipo int se tendrán menos garantías de precisión. Por ejemplo:

```
#include <iostream>
#include <math.h>
using namespace std;
main()
   {
   double x, y=3;
   for (x = pow (y, 3.0); x > 2.0; x = sqrt (x))
   {
      cout << "x es ahora igual a "<< x << endl;
   }
}
```

Es necesario incluir la librería <cmath> para poder hacer uso de funciones matemáticas. En el ejemplo anterior: sqrt - raíz cuadrada y pow - potencia.

Precauciones en el uso de for

(1) No es recomendable modificar el valor de cualquier variable de la condición del bucle dentro del cuerpo de un bucle for, ya que se pueden producir resultados imprevistos.

Ejemplo:

```
#include <iostream>
using namespace std;
main()
{
   int i, limite =1;

   for (i = 0; i <= limite; i++)
   {
      cout << i << endl;
      limite++;
   }
}
```

Este ejemplo produce una secuencia infinita de enteros ya que en cada iteración, la expresión (limite++) incrementa limite en 1, antes de que i++ incremente a i. A consecuencia de esto, la condición del bucle (i <= limite) siempre es cierta.

(2) Se ha de tener cuidado a la hora se situar el punto y coma (;) después del paréntesis final del bucle for. Por ejemplo el bucle:

```
#include <iostream>
using namespace std;
main()
{
   int i;
   for (i = 1; i <= 10; i++); // Atención al ;
   {
      cout << "Elementos de Programación" << endl;
   }
}
```

No se visualiza la frase "Elementos de Programación" 10 veces, ni se produce un mensaje de error por parte del compilador. En realidad lo que sucede es que se visualiza una vez la frase "Elementos de Programación", ya que la sentencia `for` es una sentencia vacía al terminar con un punto y coma (;). Por tanto, lo que sucede es que la sentencia `for` no hace absolutamente nada y tras 10 iteraciones, y por tanto después de que el bucle `for` haya terminado, se ejecuta la sentencia `cout` y se escribe la frase "Elementos de Programación". El siguiente bucle for implementa un bucle infinito:

```
#include <iostream>
using namespace std;
main()
{
   for (; ;)
   {
      cout << "Siempre igual" << endl;
   }
}
```

La razón es que se han eliminado la expresión de inicialización, la condición del bucle y la expresión de incremento. Por tanto, el bucle producirá la salida "Siempre igual" un número ilimitado de veces.

¿Qué estructura de iteración usar? Como recomendación :

· Si antes de la ejecución del bucle se puede conocer el número de veces que se ejecutará éste, es mejor utilizar una sentencia for.

· En otro caso y si se ejecuta al menos una vez, debemos utilizar un do-while.

· En cualquier otro caso utilizaremos un bucle while.

No se permite el uso de las sentencias break O continue en un bucle (for, do-while o while)

Capítulo 7

Procedimientos y Funciones en C++

En su concepción más simple, un procedimiento es una construcción que permite dar nombre a un conjunto de sentencias y declaraciones asociadas que se usan para resolver un subproblema dado. Los procedimientos no siempre realizan la misma función y pueden recibir parámetros. No repetir código no es la única razón para estructurar un programa usando procedimientos. Puesto que un sub-problema puede codificarse como un procedimiento, un problema complejo puede dividirse en sub-problemas más simples, quienes a su vez pueden ser de nuevo subdivididos hasta llegar a la descripción de sub-problemas muy simples que se puedan codificar como procedimientos escritos en C++. La filosofía que acabamos de presentar se denomina Refinamiento Progresivo o por pasos, diseño descendente, Programación Top-Down o bien Divide y Vencerás.

En C++ no hay diferencias entre procedimientos y funciones: todo son funciones, con la diferencia de que un procedimiento es una función que no devuelve nada (void). Tanto procedimientos como funciones, deben ser declarados antes de poder ser usados. Esto es posible de dos formas: indicando su cabecera (nombre, parámetros y tipo de retorno) después de la zona de constantes y antes del programa principal, o bien escribir el procedimiento o la función entera en dicha zona. Nosotros recomendamos la primera opción, ya que evita problemas de orden de implementación para solucionar problemas de ámbito. Mientras que un procedimiento ejecuta un grupo de sentencias, una función además devuelve un valor al punto donde se llamó. Una llamada a una función puede aparecer como operando de alguna expresión. El valor de la función se usa, por tanto, para calcular el valor total de la expresión.

Obsérvese que el tipo del resultado que devolverá la función debe de aparecer declarado en la cabecera sustituyendo a la palabra void que identifica a un procedimiento. En este sentido, en C++, puede verse a un procedimiento como un tipo especial de función que devuelve un valor void (nulo).

- Las funciones en C++ pueden devolver cualquier tipo menos arrays.
- Toda función debe ejecutar una sentencia return.

Sintaxis de un Función:

```
Cabecera:  <tipo> <identificador> ( <parámetros> ) ;
```

Ejemplo: `int mi_funcion( int x);`

En el prototipado (declaración de la función antes de la función `main`(), no es necesario que se declare la variable del parámetro, solo el tipo de dato que se espera reciba la función.

Implementación:

```
<tipo> <identificador> ( <parámetros> )
      {
         <sentencias>
         return <valor>;
      }
```

Ejemplo:

```
int mi_funcion( int x)
{
   return 0;
}
```

Paso por valor

Los parámetros de entrada (valor) se usan para proporcionar información de entrada a un procedimiento. Dentro de éste pueden considerarse como variables cuyo valor inicial es el resultado de evaluar los parámetros actuales. Como parámetro actual debe aparecer una expresión cuyo resultado sea un valor de un tipo asignable al correspondiente parámetro formal. Puesto que las variables usadas como parámetros formales de entrada no sirven para cambiar a los parámetros actuales (sólo para conocer su valor en el momento de la llamada y asignarle un nombre a ese valor dentro del procedimiento) se les suele denominar Parámetros por valor. Ejemplo:

```
void dibLineas( int anchura, int altura )
{
   int nFila;
   int nColumna;
   for( nFila = 1 ; nFila <= altura; ++nFila )
   {
      for( nColumna = 1; nColumna <= anchura; ++nColumna )
      {
        cout << "-";
      }
      cout << endl;
   }
}
```

Paso por referencia

A diferencia del paso por valor, el paso por referencia permite que la función invocada modifique el valor original de las variables usadas como argumentos.

En C++, el paso de parámetros por referencia se realiza creando un alias del identificador de la variable que forma el parámetro actual,

```
#include <iostream>
using namespace std;
main()
{   int i=3, j=50;
    cout>>"i">>i>>"j">>j;
    intercambio(&i, &j);
    cout>>"i">>i>>"j">>j;
    return 0;
}
void intercambio(int* a, int* b)
{    int aux = *a;
     *a = *b;
     *b = aux;
}
```

Ambito (alcance)

El ámbito es una zona del programa en el cual una variable es visible o en pocas palabras puede ser utilizada.

Existen 4 tipos de ámbitos:

- Programa
- Archivo fuente
- Función
- Bloque

```
int i;                  //Ambito de programa
static int j;           //Ambito de archivo
float func(int k)       //Ambito de función
{
   {
       int m;           //Ambito de bloque
   }
}
```

Operador de acceso a ámbito

Para el acceso a las variables definidas en los ámbitos, se utiliza el operador de ámbito, dado que en C++ existen múltiples espacios de nombres. Éste operador debe debe su nombre al hecho de que sirve como puente o camino para acceder a objetos que momentáneamente están fuera de ámbito.

El operador :: de acceso a ámbito o resolución de ámbito, permite referenciar (acceder) a un tipo, objeto, función o enumerador incluso si su identificador está oculto. Por ejemplo, anteponiéndolo a un identificador global, permite acceder al mismo incluso cuando es ocultado por una redeclaración homónima.

```
include<iostream>
int x = 2;              // Variable global...
main() {
   int x = 3;           // Variable local
   std::cout << "X = "<< x   << std::endl; // L.5: x local
   std::cout << "X = "<< ::x << std::endl; // L.6: x global
}
```

El mismo ejemplo con el uso del espacio de nombre standard

```
#include<iostream>
using namespaces std;
int x = 2;              // Variable global...
main() {
   int x = 3;           // Variable local
   cout << "X = "<< x   << endl; // L.5: x local
   cout << "X = "<< ::x << endl; // L.6: x global
}
```

Capítulo 8

Recursividad

Se dice que una función es recursiva cuando se define en función de si misma.

No todas la funciones pueden llamarse a si mismas, sino que deben estar diseñadas especialmente para que sean recursivas, de otro modo podrían conducir a bucles infinitos, o a que el programa termine inadecuadamente.

Tampoco todos los lenguajes de programación permiten usar recursividad.

C++ permite la recursividad. Cada vez que se llama a una función, se crea un juego de variables locales, de este modo, si la función hace una llamada a si misma, se guardan sus variables y parámetros, usando la pila, y la nueva instancia de la función trabajará con su propia copia de las variables locales. Cuando esta segunda instancia de la función retorna, recupera las variables y los parámetros de la pila y continua la ejecución en el punto en que había sido llamada.

Por ejemplo:

Prodríamos crear una función recursiva para calcular el factorial de un número entero.

El factorial se simboliza como n!, se lee como "n factorial", y la definición es:

n! = n * (n-1) * (n-2) * ... * 1

Hay algunas limitaciones:

No es posible calcular el factorial de números negativos, no está definido.

El factorial de cero es 1.

De modo que una función bien hecha para cálculo de factoriales debería incluir un control para esos casos:

```
/* Función recursiva para cálculo de factoriales */
int factorial(int n) {
   if(n < 0) return 0;
   else if(n > 1) return n*factorial(n-1); //Recursividad
   return 1; /* Condición de terminación, n == 1 */
}
```

Análisis paso a paso del ejemplo: factorial(4):

1a Instancia

n=4

n > 1

salida ← 4 * factorial(3) (Guarda el valor de n = 4)

2a Instancia

n > 1

salida ← 3*factorial(2) (Guarda el valor de n = 3)

3a Instancia

n > 1

salida ← 2*factorial(1) (Guarda el valor de n = 2)

4a Instancia

n == 1 → retorna 1

3a Instancia

(recupera n=2 de la pila) retorna 1*2=2

2a instancia

(recupera n=3 de la pila) retorna 2*3=6

1a instancia

(recupera n=4 de la pila) retorna 6*4=24

Valor de retorno → 24

Aunque la función factorial es un buen ejemplo para demostrar cómo funciona una función recursiva, la recursividad no es un buen modo de resolver esta función, que sería más sencilla y rápida con un simple bucle for.

La recursividad consume muchos recursos de memoria y tiempo de ejecución, y se debe aplicar a funciones que realmente le saquen partido.

Capítulo 9

Arrays

Un arreglo en C++ es un conjunto de datos que se almacenan en memoria de manera contigua con el mismo nombre. Para diferenciar los elementos de un arreglo se utilizan índices detrás del nombre del arreglo y encerrados por []. El elemento 5° (quinto) de un arreglo, es representado por el índice [4], ya que los índices comienzan en 0. Esto significa que un arreglo de 10 elementos tendría los índices del 0 al 9: [0...9]

Un arreglo o array (en inglés) es una colección de variables relacionadas a las que se hace referencia por medio de un nombre común. Otra definición válida es que un arreglo es un conjunto de datos que se almacenan en memoria de manera contigua con el mismo nombre y para diferenciar los elementos de un arreglo se utiliza un índice. En el lenguaje C++ un arreglo se le conoce como un tipo de dato compuesto.

Array es un término del Inglés que no se suele traducir. El término arreglo no es en realidad una traducción, sino un anglicismo (más bien, un término que no existe en español). Podríamos traducir array como colección, selección, o tal vez mejor, como formación. En este curso, de todos modos, usaremos el término array.

Sintaxis:

```
<tipo> <identificador>[<núm_elemen>][[<núm_elemen>]...];
```

Un detalle muy importante es que los valores para el número de elementos deben ser constantes enteras.

Se pueden usar tantas dimensiones (índices) como queramos, el límite lo impone sólo la cantidad de memoria disponible. Cuando sólo se usa un índice se suele hablar de vectores, cuando se usan dos, de matrices.

C++ no verifica el ámbito de los índices. Para poder hacerlo, el compilador tendría que agregar código, ya que los índices pueden ser variables, su valor debe ser verificado durante la ejecución, no durante la compilación. Esto está en contra de la filosofía de C++ de crear programas compactos y rápidos. Así que es tarea nuestra asegurarnos de que los índices están dentro de los márgenes correctos.

Si se declara un array de 10 elementos, no se obtienen errores al acceder al elemento 11, las operaciones de lectura no son demasiado peligrosas, al menos en la mayoría de los casos. Sin embargo, si se asignan valores a elementos fuera del ámbito declarado, se estará accediendo a zonas de memoria que pueden pertenecer a otras variables o incluso al código ejecutable del propio programa, con consecuencias generalmente desastrosas.

Los arrays pueden ser inicializados en la declaración.

Ejemplos:

```
float R[10] = {2, 32, 4.6, 2, 1, 0.5, 3, 8, 0, 12};
float S[] = {2, 32, 4.6, 2, 1, 0.5, 3, 8, 0, 12};
int N[] = {1, 2, 3, 6};
int M[][3] = { 213, 32, 32, 32, 43, 32, 3, 43, 21};
```

Cuando se inicializan los arrays en la declaración no es obligatorio especificar el tamaño para la primera dimensión, como ocurre en los ejemplos de las líneas 2, 3 y 4

En estos casos la dimensión que queda indefinida se calcula a partir del número de elementos en la lista de valores iniciales. El compilador sabe contar y puede calcular el tamaño necesario de la dimensión para contener el número de elementos especificados.

En el caso 2, el número de elementos es 10, ya que hay diez valores en la lista.

En el caso 3, será 4.

En el caso 4, será 3, ya que hay 9 valores, y la segunda dimensión es 3: 9/3=3.

Operadores con arrays

Ya se ha visto que se puede usar el operador de asignación con arrays, el otro operador que tiene sentido con los arrays es sizeof.

Aplicado a un array, el operador sizeof devuelve el tamaño de todo el array en bytes. Se Puede obtener el número de elementos, de ser necesario, dividiendo ese valor entre el tamaño de uno de los elementos.

```
int main()
{
   int array[231];
   int nElementos;
   nElementos = sizeof(array)/sizeof(int);
   nElementos = sizeof(array)/sizeof(array[0]);
   return 0;
}
```

Las dos formas son válidas, pero la segunda es, tal vez, más general.

La utilidad de esta técnica es, como mucho, limitada. Desde el momento en que se deben usar constantes al declarar los arrays, su tamaño es siempre conocido, y por lo tanto, su cálculo es predecible.

Representación de un arreglo de una dimensión (vector):

1	arreglo[0]
2	arreglo[1]
3	arreglo[2]
4	arreglo[3]
5	arreglo[4]
6	arreglo[5]

Este vector se declara:

```
float arreglo[6]
```

Un elemento del arreglo se accede indexando el arreglo por medio de un número del elemento. En C++ todos los arreglos empiezan en 0, esto quiere decir que si se desea acceder al primer elemento del arreglo se debe usar el índice igual a 0. Para indexar un arreglo se especifica el índice del elemento que interesa dentro de un corchete, ejemplo;

```
valor = arreglo[1];
```

Los arreglos empiezan en 0, de manera que el índice 1 se refiere al segundo elemento. Para asignar el valor a un elemento de un arreglo, es necesario colocar el elemento en el lado izquierdo de una sentencia de asignación.

```
mi_arreglo[0] = 100;
```

C++ almacena arreglos de una sola dimensión en una localización de memoria contigua con el primer elemento en la posición más baja. De esta manera, `mi_arreglo[0]` es adyacente a `mi_arreglo[1]`, que es adyacente a `mi_arreglo[2]` y así sucesivamente.

Es válido utilizar el valor de un elemento de un arreglo donde quiera que uno usaría una variable sencilla o una constante.

Ejemplo:

Cargar el arreglo sqrs con los cuadrados de los números del 1 al 10 y luego visualizarlos.

```
using namespace std;
#include <iostream>
int main()
{
  int sqrs[10];
  int i;
  for (i=1;i<11;i++) {
      sqrs[i-1]=i*i;
      }
  for (i=0;i<10;i++) {
      cout<<sqrs[i]<<endl;
      }
  return 0;
}
```

Matriz bidimensional

En la siguiente tabla se muestra cual sería la forma y los elementos de matriz[4][5], (array de tamaño 4×5 con el nombre de variable matriz). Véase que el primer elemento del array bidimensional es el matriz[0][0], el siguiente sería el matriz[0][1], y así, hasta llegar al elemento matriz[3][4]:

Matriz bidimensional						
Elemento en		**Índice de columna**				
		0	**1**	**2**	**3**	**4**
Índice de fila	**0**	matriz [0][0]	matriz [0][1]	matriz [0][2]	matriz [0][3]	matriz [0][4]
	1	matriz [1][0]	matriz [1][1]	matriz [1][2]	matriz [1][3]	matriz [0][4]
	2	matriz [2][0]	matriz [2][1]	matriz [2][2]	matriz [2][3]	matriz [0][4]
	3	matriz [3][0]	matriz [3][1]	matriz [3][2]	matriz [3][3]	matriz [0][4]

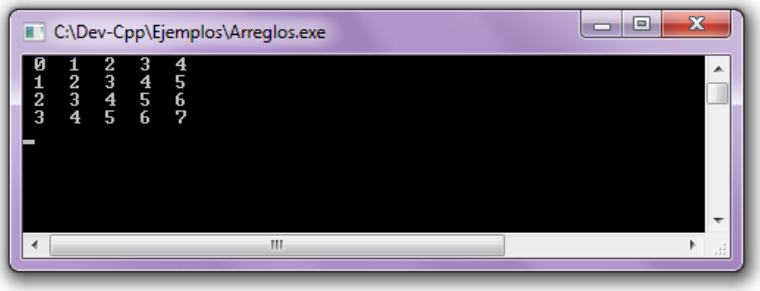
Iteraciones dentro de un array (matriz)

Con el fin de leer o escribir sobre cada uno de los componentes de una matriz se deben crear dos ciclos de iteración. Ejemplo:

```
#include <iostream>
using namespace std;
main()
{
      int matriz[4][5], i , j;
      for (i=0 ; i<4 ; i++)
      {
          for (j=0 ; j<5 ; j++)
          {
              matriz[i][j] = i + j ;
              cout<<matriz[i][j]<<" ";
          }
          cout<<endl;
      }
      system("pause");
}
```

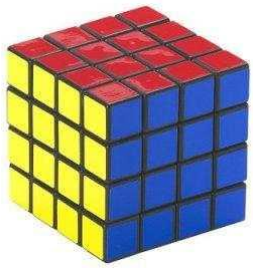
En este caso se utilizan dos variables, una para iterar sobre las filas “i” y otra para iterar sobre las columnas “j” de la matriz.

Como se suman los valores de “i” y “j”, entonces el resultado en pantalla sería el siguiente:



Por otra parte, en lenguaje C++ las matrices se almacenan en memoria "por filas", es decir, los elementos de la fila primera (de índice 0) van consecutivos y cuando acaba el último de la primera fila empieza a almacenarse el primero de la segunda, y así sucesivamente hasta llegar a la última fila.

Para ejemplificar esto considerar la siguiente figura (cubo mágico):



Una matriz bidimensional (AxB) representaría solo una cara del cubo mágico.

Otros ejemplos de declaración de matrices bidimensionales:

```
int dias[2][12] = {{31,28,31,30,31,30,31,31,30,31,30,31},
                   {31,29,31,30,31,30,31,31,30,31,30,31} };
char lista[5][4] = { "Cat", "Bar", "Cab", "Lab", "Tab" };
```

Para arreglos bidimensionales, basta con recordar que primero están los elementos de la fila 0, enseguida los de la fila 1, y así sucesivamente. La inicialización:

```
double a[2][3] = { 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
```

Produce el mismo resultado que:

```
double a[2][3];
a[0][0] = 1.1;
a[0][1] = 1.2;
a[0][2] = 1.3;
a[1][0] = 1.4;
a[1][1] = 1.5;
a[1][2] = 1.6;
```

La siguiente inicialización también hubiera producido el mismo resultado anterior:

```
double a[][3] = { 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
```

Matrices multidimensionales:

Sintaxis:

La forma de declararlas es mediante una serie de expresiones entre corchetes [ ]:

```
tipoX etiqueta [<expr-c1>][<expr-c2>]...[exp-cn]
```

Ejemplo:

```
int matriz[4][4][4];
```

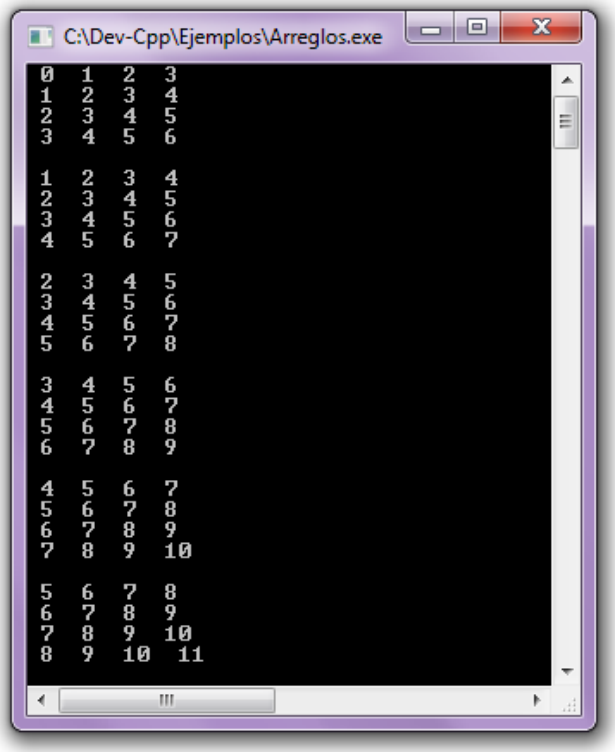
El resultado de las expresiones <expr-c> representan el tamaño de cada submatriz, y deben reducirse a enteros positivos (unsigned int) distintos de cero.

Cada submatriz es una "dimensión" de la matriz total. En el caso de matrices de dos dimensiones, <expr-c1> y <expr-c2> se denominan respectivamente filas y columnas. En el caso de más de dos dimensiones, la expresión <expr-cn> se denomina sencillamente "dimensión-n".

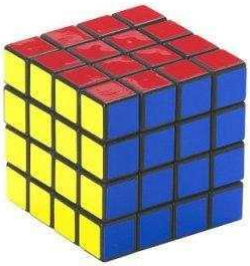
Ejemplo:

```
#include <iostream>
using namespace std;
main()
{
      int matriz[6][4][4], i, j, k;
      for (i=0;i<6;i++)
      {
          for(j=0;j<4;j++)
          {
                          for (k=0;k<4;k++)
                          {
                              matriz[i][j][k] = i+j+k;
                              cout<<matriz[i][j][k]<<"  ";
                          }
                          cout<<endl;
          }
          cout<<endl;
      }
      system("pause");
}
```

El resultado en pantalla sería el siguiente:



Para ejemplificar esto considerar nuevamente el siguiente cubo mágico:



Una matriz bidimensional (AxB) representaría solo una cara del cubo mágico, una matriz de tres dimensiones (AxBxC) representaría todo el cubo mágico.

Ejercicio de refuerzo para matriz bidimensional:

Imprimir la siguiente tabla con matrices bidimensionales:

	0	1	2	3	4
0	1	2	3	4	5
1	2	4	6	8	10
2	3	6	9	12	15

```
#include <iostream>
using namespace std;
int matriz[3][5];
int n,m;
main()
{
      for (n=0;n<3;n++)
      {
          for (m=0;m<5;m++)
          {
              matriz[n][m]=(n+1)*(m+1);
          }
      }
      for (n=0;n<3;n++)
      {
          for (m=0;m<5;m++)
          {
              cout<<matriz[n][m]<<"  ";
          }
          cout<<endl;
      }
      system("pause");
}
```

Capítulo 10

Algoritmos de Ordenación y Búsqueda

Una de las tareas que realizan las computadoras en el procesamiento de datos es la ordenación. El estudio de diferentes métodos de ordenación es una tarea intrínsecamente interesante desde un punto de vista teórico y, naturalmente, práctico. El capítulo estudia los algoritmos y técnicas de ordenación más usuales y su implementación en C++. De igual modo se estudiará el análisis de los diferentes métodos de ordenación con el objeto de conseguir la máxima eficiencia en su uso real.

Conceptos Preliminares.

Antes de comenzar a ver cada algoritmo vamos a ponernos de acuerdo en algunos conceptos, para que no haya confusiones:

Clave: La parte de un registro por la cual se ordena la lista. Por ejemplo, una lista de registros con campos nombre, direccion y telefono se puede ordenar alfabéticamente de acuerdo a la clave nombre. En este caso los campos direccion y telefono no se toman en cuenta en el ordenamiento.

Criterio de ordenamiento (o de comparación): EL criterio que utilizamos para asignar valores a los registros con base en una o más claves. De esta manera decidimos si un registro es mayor o menor que otro. En el pseudocódigo presentado más adelante simplemente se utilizarán los símbolos < y >, para mayor simplicidad.

Registro: Un grupo de datos que forman la lista. Pueden ser datos atómicos (enteros, caracteres, reales, etc.) o grupos de ellos, que en C equivalen a las estructuras.

Cuando se estudian algoritmos de todo tipo, no sólo de ordenamiento, es bueno tener una forma de evaluarlos antes de pasarlos a código, que se base en aspectos independientes de la plataforma o el lenguaje. De esta manera podremos decidir cuál se adapta mejor a los requerimientos de nuestro programa. Así que veamos estos aspectos:

Estabilidad: Cómo se comporta con registros que tienen claves iguales. Algunos algoritmos mantienen el orden relativo entre éstos y otros no. Veamos un ejemplo. Si tenemos la siguiente lista de datos (nombre, edad): "Pedro 19, Juan 23, Felipe 15, Marcela 20, Juan 18, Marcela 17", y la ordenamos alfabéticamente por el nombre con un algoritmo estable quedaría así: "Felipe 15, Marcela 20, Marcela 17, Juan 23, Juan 18, Pedro 19". Un algoritmo no estable podría dejar a Juan 18 antes de Juan 23, o a Marcela 20 después de Marcela 17.

Tiempo de ejecución: La complejidad del algoritmo, que no tiene que ver con dificultad, sino con rendimiento. Es una función independiente de la implementación. Te la voy a explicar brevemente: tenemos que identificar una operación fundamental que realice nuestro algoritmo, que en este caso es comparar. Ahora contamos cuántas veces el algoritmo necesita comparar. Si en una lista de n términos realiza n comparaciones la complejidad es O(n). (En realidad es un poco más complicado que eso, pero lo vamos a hacer así: recuerda que dije que te iba a explicar brevemente). Algunos ejemplos de complejidades comunes son:

O(1) : Complejidad constante.

O(n2) : Complejidad cuadrática.

O(n log(n)) : Complejidad logarítmica.

Un algoritmo de complejidad O(n) es más rápido que uno de complejidad O(n2). Otro aspecto a considerar es la diferencia entre el peor y el mejor caso. Cada algoritmo se comporta de modo diferente de acuerdo a cómo se le entregue la información; por eso es conveniente estudiar su comportamiento en casos extremos, como cuando los datos están prácticamente ordenados o muy desordenados.

Requerimientos de memoria: El algoritmo puede necesitar memoria adicional para realizar su labor. En general es preferible que no sea así, pero es común en la programación tener que sacrificar memoria por rendimiento.

Algoritmos más comunes.

La siguiente es una tabla comparativa de algunos algoritmos de ordenamiento.

Tabla comparativa de algoritmos de ordenamiento mas utilizados

Nombre	Complejidad	Estabilidad	Memoria adicional
Ordenamiento Burbuja	$O(n^2)$	Estable	No
Ordenamiento por Selección	$O(n^2)$	No Estable	No
Ordenamiento por Inserción	$O(n^2)$	Estable	No
Ordenamiento Rápido (Quicksort)	$O(n * \log_2(n))$	No Estable	No

Ordenamiento Burbuja (Bubblesort)

Descripción: Este probablemente sea el algoritmo más sencillo. Ideal para empezar. Consiste en ciclar repetidamente a través de la lista, comparando elementos adyacentes de dos en dos. Si un elemento es mayor que el que está en la siguiente posición se intercambian.

Tabla de variables		
Nombre	**Tipo**	**Uso**
lista	Cualquiera	Lista a ordenar
TAM	Constante entera	Tamaño de la lista
i	Entero	Contador
j	Entero	Contador
temp	El mismo que los elementos de la lista	Para realizar los intercambios

```
1. for (i=1; i<TAM; i++)
2.      for j=0 ; j<TAM - 1; j++)
3.           if (lista[j] > lista[j+1])
4.                temp = lista[j];
5.                lista[j] = lista[j+1];
6.                lista[j+1] = temp;
```

Sea la lista:

4 - 3 - 5 - 2 - 1

Existen 5 elementos. Es decir, TAM toma el valor 5. Inicialmente se comparan el primero con el segundo elemento. 4 es mayor que 3, así que son intercambiados. Ahora se tiene la lista:

3 - 4 - 5 - 2 - 1

Se compara el segundo con el tercero: 4 es menor que 5, así que no se hace nada. Se continua con el tercero y el cuarto: 5 es mayor que 2, así que son intercambiados. Ahora se tiene la lista:

3 - 4 - 2 - 5 - 1

Se compara el cuarto y el quinto: 5 es mayor que 1. Se intercambian nuevamente:

3 - 4 - 2 - 1 - 5

Repitiendo este proceso se obtenien los siguientes resultados:

3 - 2 - 1 - 4 - 5

2 - 1 - 3 - 4 - 5

1 - 2 - 3 - 4 – 5

Optimizando.

Se pueden realizar algunos cambios en este algoritmo que pueden mejorar su rendimiento.

En cada pasada a través de la lista, un elemento va quedando en su posición final. Así en la primera pasada el 5 (elemento mayor) queda en la última posición, en la segunda el 4 (el segundo mayor elemento) queda en la penúltima posición. Se puede evitar hacer comparaciones innecesarias al disminuir el número de éstas en cada pasada. Se debe cambiar el ciclo interno de esta manera:

```
for (j=0; j<TAM - i; j++)
```

Puede ser que los datos queden ordenados antes de completar el ciclo externo. Podemos modificar el algoritmo para que verifique si se han realizado intercambios. Si no se han hecho entonces terminamos con la ejecución, pues eso significa que los datos ya están ordenados.

Otra forma es ir guardando la última posición en que se hizo un intercambio, y en la siguiente pasada sólo comparar hasta antes de esa posición.

Análisis del algoritmo.

Éste es el análisis para la versión no optimizada del algoritmo:

Estabilidad: Este algoritmo nunca intercambia registros con claves iguales. Por lo tanto es estable.

Requerimientos de Memoria: Este algoritmo sólo requiere de una variable adicional para realizar los intercambios.

Tiempo de Ejecución: El ciclo interno se ejecuta n veces para una lista de n elementos. El ciclo externo también se ejecuta n veces. Es decir, la complejidad es n * n = O(n2). El comportamiento del caso promedio depende del orden de entrada de los datos, pero es sólo un poco mejor que el del peor caso, y sigue siendo O(n2).

Ventajas:

Fácil implementación.

No requiere memoria adicional.

Desventajas:

Muy lento.

Realiza numerosas comparaciones.

Realiza numerosos intercambios.

Este algoritmo es uno de los más pobres en rendimiento. No se recomienda su uso. Está aquí con fines académicos.

Ordenamiento por Selección.

Descripción.

Este algoritmo también es sencillo. Consiste en lo siguiente:

Buscar el elemento más pequeño de la lista.

Intercambiar con el elemento ubicado en la primera posición de la lista.

Buscar el segundo elemento más pequeño de la lista.

Intercambiar con el elemento que ocupa la segunda posición en la lista.

Repitir este proceso hasta que se haya ordenado toda la lista.

Tabla de variables

Nombre	Tipo	Uso
lista	Cualquiera	Lista a ordenar
TAM	Constante entera	Tamaño de la lista
i	Entero	Contador
pos_men	Entero	Posición del menor elemento de la lista
temp	El mismo que los elementos de la lista	Para realizar los intercambios

```
1. for (i=0; i<TAM - 1; i++)
2.      pos_men = Menor(lista, TAM, i);
3.      temp = lista[i];
4.      lista[i] = lista [pos_men];
5.      lista [pos_men] = temp;
```

Observación: Menor(lista, TAM, i) es una función que busca el menor elemento entre las posiciones i y TAM-1. La búsqueda es lineal (elemento por elemento).

Un ejemplo.

Ordenar la lista (la misma del ejemplo anterior) :

4 - 3 - 5 - 2 - 1

Buscar el elemento menor entre la primera y última posición. Es el 1. Intercambiar con el 4 y la lista quedará así:

1 - 3 - 5 - 2 - 4

Buscamos el menor elemento entre la segunda y la última posición. Es el 2. Intercambiar con el elemento en la segunda posición, es decir el 3. La lista quedará así:

1 - 2 - 5 - 3 - 4

Buscar el menor elemento entre la tercera posición y la última. Es el 3, que se intercambia con el 5:

1 - 2 - 3 - 5 - 4

El menor elemento entre la cuarta y quinta posición es el 4, que se intercambia con el 5:

1 - 2 - 3 - 4 - 5

Fin del ciclo.

Análisis del algoritmo.

Estabilidad: No es estable.

Requerimientos de Memoria: Al igual que el ordenamiento burbuja, este algoritmo sólo necesita una variable adicional para realizar los intercambios.

Tiempo de Ejecución: El ciclo externo se ejecuta n veces para una lista de n elementos. Cada búsqueda requiere comparar todos los elementos no clasificados. Luego la complejidad es O(n2). Este algoritmo presenta un comportamiento constante independiente del orden de los datos. Luego la complejidad promedio es también O(n2).

Ventajas:

Fácil implementación.

No requiere memoria adicional.

Realiza pocos intercambios.

Rendimiento constante: poca diferencia entre el peor y el mejor caso.

Desventajas:

Lento.

Realiza numerosas comparaciones.

Este es un algoritmo lento. No obstante, ya que sólo realiza un intercambio en cada ejecución del ciclo externo, puede ser una buena opción para listas con registros grandes y claves pequeñas.

Ordenamiento por Inserción

Descripción.

Este algoritmo también es bastante sencillo. Es el utilizado por cualquier jugador de cartas al recibirlas y empezar su ordenación: Se toma la primera y se la coloca en la mano. Luego se toma la segunda y se la compara con la que ya está en la mano, si es mayor, se la coloca a la derecha, y si es menor a la izquierda. Después se toma la tercera y se la compara con las que ya se tienen en la mano, desplazándola hasta que quede en su posición final. Continúo haciendo esto, insertando cada carta en la posición que le corresponde, hasta que las tengo todas en orden.

Para simular esto en un programa necesitamos tener en cuenta algo: no podemos desplazar los elementos así como así o se perderá un elemento. Lo que hacemos es guardar una copia del elemento actual (que sería como la carta que tomamos) y desplazar todos los elementos mayores hacia la derecha. Luego copiamos el elemento guardado en la posición del último elemento que se desplazó.

Tabla de variables

Nombre	Tipo	Uso
lista	Cualquiera	Lista a ordenar
TAM	Constante Entera	Tamaño de la lista
i	Entero	Contador
j	Entero	Contador
temp	El mismo que los elementos de la lista	Para realizar los intercambios

```
1. for (i=1; i<TAM; i++)
2.      temp = lista[i];
3.      j = i - 1;
4.      while ( (lista[j] > temp) && (j >= 0) )
5.           lista[j+1] = lista[j];
6.           j--;
7.      lista[j+1] = temp;
```

Nota: Observa que en cada iteración del ciclo externo los elementos 0 a i forman una lista ordenada.

Un ejemplo

Sea la lista:

4 - 3 - 5 - 2 - 1

temp toma el valor del segundo elemento, 3. La primera "carta" es el 4. Ahora se compara: 3 es menor que 4. Luego se desplaza el 4 una posición a la derecha y después se copia el 3 en el lugar ocupado anteriormente por el 4.

4 - 4 - 5 - 2 - 1

3 - 4 - 5 - 2 - 1

El siguiente elemento es 5. Se compara con 4. Es mayor que 4, así que no ocurren intercambios.

Se continua con el 2. Es menor que cinco: se desplaza el 5 una posición a la derecha:

3 - 4 - 5 - 5 - 1

Se compara con 4: es menor, así que se desplaza el 4 una posición a la derecha:

3 - 4 - 4 - 5 - 1

Se compara con 3. Se desplaza el 3 una posición a la derecha:

3 - 3 - 4 - 5 - 1

Finalmente se copia el 2 en su posición final:

2 - 3 - 4 - 5 - 1

El último elemento a ordenar es el 1. Cinco es menor que 1, así que se lo desplaza una posición a la derecha:

2 - 3 - 4 - 5 - 5

Continuando con el procedimiento la lista va quedando así:

2 - 3 - 4 - 4 - 5

2 - 3 - 3 - 4 - 5

2 - 2 - 3 - 4 - 5

1 - 2 - 3 - 4 - 5

Análisis del algoritmo.

Estabilidad: Este algoritmo nunca intercambia registros con claves iguales. Por lo tanto es estable.

Requerimientos de Memoria: Solo una variable adicional para realizar los intercambios.

Tiempo de Ejecución: Para una lista de n elementos el ciclo externo se ejecuta n-1 veces. El ciclo interno se ejecuta como máximo una vez en la primera iteración, 2 veces en la segunda, 3 veces en la tercera, etc. Esto produce una complejidad O(n2).

Ventajas:

Fácil implementación.

Requerimientos mínimos de memoria.

Desventajas:

Lento.

Realiza numerosas comparaciones.

Este también es un algoritmo lento, pero puede ser de utilidad para listas que están ordenadas o semi-ordenadas, porque en ese caso realiza muy pocos desplazamientos.

Ordenamiento Rápido (Quicksort)

Descripción.

Esta es probablemente la técnica más rápida conocida. Fue desarrollada por C.A.R. Hoare en 1960. El algoritmo original es recursivo, pero se utilizan versiones iterativas para mejorar su rendimiento (los algoritmos recursivos son en general más lentos que los iterativos, y consumen más recursos). El algoritmo fundamental es el siguiente:

Se selecciona un elemento de la lista. Puede ser cualquiera y se lo denomina elemento de división se busca la posición que le corresponde en la lista ordenada (explicado más abajo).

Se acomodan los elementos de la lista a cada lado del elemento de división, de manera que a un lado queden todos los menores que él y al otro los mayores (explicado más abajo también). En este momento el elemento de división separa la lista en dos sub-listas (de ahí su nombre).

Realizas esto de forma recursiva para cada sub-lista mientras éstas tengan un largo mayor que 1 (uno) . Una vez terminado este proceso todos los elementos estarán ordenados.

Una idea preliminar para ubicar el elemento de división en su posición final sería contar la cantidad de elementos menores y colocarlo un lugar más arriba. Pero luego habría que mover todos estos elementos a la izquierda del elemento, para que se cumpla la condición y pueda aplicarse la recursividad. Reflexionando un poco más se obtiene un procedimiento mucho más efectivo. Se utilizan dos índices: i, al contador por la izquierda, y j, al contador por la derecha. El algoritmo es éste:

Se recorres la lista simultáneamente con i y j: por la izquierda con i (desde el primer elemento), y por la derecha con j (desde el último elemento).

Cuando lista[i] sea mayor que el elemento de división y lista[j] sea menor se los intercambia.

Se repite esto hasta que se crucen los índices.

El punto en que se cruzan los índices es la posición adecuada para colocar el elemento de división, porque es sabido que a un lado los elementos son todos menores y al otro son todos mayores (o habrían sido intercambiados).

Al finalizar este procedimiento el elemento de división queda en una posición en que todos los elementos a su izquierda son menores que él, y los que están a su derecha son mayores.

Tabla de variables

Nombre	Tipo	Uso
lista	Cualquiera	Lista a ordenar
inf	Entero	Elemento inferior de la lista
sup	Entero	Elemento superior de la lista
elem_div	El mismo que los elementos de la lista	El elemento divisor
temp	El mismo que los elementos de la lista	Para realizar los intercambios
i	Entero	Contador por la izquierda
j	Entero	Contador por la derecha
cont	Entero	El ciclo continua mientras cont tenga el valor 1

```
Nombre Procedimiento: OrdRap
Parámetros:
   lista a ordenar (lista)
   índice inferior (inf)
   índice superior (sup)

   // Inicialización de variables
   1. elem_div = lista[sup];
   2. i = inf - 1;
   3. j = sup;
   4. cont = 1;

   // Verificamos que no se crucen los límites
   5. if (inf >= sup)
   6.      retornar;

   //  Clasificamos la sublista
   7. while (cont)
   8.      while (lista[++i] < elem_div);
   9.      while (lista[--j] > elem_div);
  10.      if (i < j)
  11.           temp = lista[i];
  12.           lista[i] = lista[j];
  13.           lista[j] = temp;
  14.      else
  15.           cont = 0;
```

```
// Copiamos el elemento de división
// en su posición final
16. temp = lista[i];
17. lista[i] = lista[sup];
18. lista[sup] = temp;

// Aplicamos el procedimiento
// recursivamente a cada sublista
19. OrdRap (lista, inf, i - 1);
20. OrdRap (lista, i + 1, sup);
```

Nota: La primera llamada debería ser con la lista, cero (0) y el tamaño de la lista menos 1 como parámetros.

Un ejemplo

Sea la lista:

5 - 3 - 7 - 6 - 2 - 1 - 4

Se comienza con la lista completa. El elemento divisor será el 4:

5 - 3 - 7 - 6 - 2 - 1 - 4

Se compara con el 5 por la izquierda y el 1 por la derecha.

5 - 3 - 7 - 6 - 2 - 1 - 4

5 es mayor que cuatro y 1 es menor. Se Intercambia:

1 - 3 - 7 - 6 - 2 - 5 - 4

Avanzando por la izquierda y la derecha:

1 - 3 - 7 - 6 - 2 - 5 - 4

3 es menor que 4: avanzando por la izquierda. 2 es menor que 4: se mantiene ahí.

1 - 3 - 7 - 6 - 2 - 5 - 4

7 es mayor que 4 y 2 es menor: Se intercambian.

1 - 3 - 2 - 6 - 7 - 5 - 4

Se avanza por ambos lados:

1 - 3 - 2 - 6 - 7 - 5 – 4

En este momento termina el ciclo principal, porque los índices se cruzaron. Ahora se intercambia lista[i] con lista[sup] (pasos 16-18):

1 - 3 - 2 - 4 - 7 - 5 - 6

Se aplicamos recursivamente a la sub-lista de la izquierda (índices 0 - 2). Se tiene lo siguiente:

1 - 3 - 2

1 es menor que 2: se avanza por la izquierda. 3 es mayor: se avanza por la derecha. Como se intercambiaron los índices termina el ciclo. Se intercambia lista[i] con lista[sup]:

1 - 2 - 3

Al llamar recursivamente para cada nueva sub-lista (lista[0]-lista[0] y lista[2]-lista[2]) se retorna sin hacer cambios (condición 5.).Para resumir, así va quedando la lista:

Segunda sub-lista: lista[4]-lista[6]

7 - 5 - 6

5 - 7 - 6

5 - 6 - 7

Para cada nueva sub-lista se retorna sin hacer cambios (se cruzan los índices).

Finalmente, al retornar de la primera llamada se tiene el arreglo ordenado:

1 - 2 - 3 - 4 - 5 - 6 - 7

Optimizando.

Hacer una versión iterativa: Para ello se utiliza una pila en que se van guardando los límites superior e inferior de cada sub-lista.

No clasificar todas las sub-listas: Cuando el largo de las sub-listas va disminuyendo, el proceso se va encareciendo. Para solucionarlo sólo se clasifican las listas que tengan un largo menor que n. Al terminar la clasificación se llama a otro algoritmo de ordenamiento que termine la labor. El indicado es uno que se comporte bien con listas casi ordenadas, como el ordenamiento por inserción por ejemplo. La elección de n depende de varios factores, pero un valor entre 10 y 25 es adecuado.

Elección del elemento de división: Se elige desde un conjunto de tres elementos: lista[inferior], lista[mitad] y lista[superior]. El elemento elegido es el que tenga el valor medio según el criterio de comparación. Esto evita el comportamiento degenerado cuando la lista está prácticamente ordenada.

Análisis del algoritmo.

Estabilidad: No es estable.

Requerimientos de Memoria: No requiere memoria adicional en su forma recursiva. En su forma iterativa la necesita para la pila.

Tiempo de Ejecución: Caso promedio. La complejidad para dividir una lista de n es O(n). Cada sub-lista genera en promedio dos sub-listas más de largo n/2. Por lo tanto la complejidad se define en forma recurrente como:

$f(1) = 1$

$f(n) = n + 2 f(n/2)$

La forma cerrada de esta expresión es: $f(n) = n \log_2 n$ Es decir, la complejidad es $O(n \log_2 n)$.

El peor caso ocurre cuando la lista ya está ordenada, porque cada llamada genera sólo una sub-lista (todos los elementos son menores que el elemento de división). En este caso el rendimiento se degrada a $O(n^2)$. Con las optimizaciones mencionadas arriba puede evitarse este comportamiento.

Ventajas:

Muy rápido

No requiere memoria adicional.

Desventajas:

Implementación un poco más complicada.

Recursividad (utiliza muchos recursos).

Mucha diferencia entre el peor y el mejor caso.

La mayoría de los problemas de rendimiento se pueden solucionar con las optimizaciones mencionadas arriba. Este es un algoritmo que se puede utilizar en la vida real. Es muy eficiente. En general será la mejor opción.

Eligiendo el algoritmo de ordenación más adecuado.

¿Cómo saber cuál es el que un programador necesita? ¿Cuál es EL algoritmo?

Cada algoritmo se comporta de modo diferente de acuerdo a la cantidad y la forma en que se le presenten los datos, entre otras cosas. No existe EL algoritmo de ordenamiento. Sólo existe el mejor para cada caso particular. Se debes conocer a fondo el problema que uno quiere resolver, y aplicar el más adecuado.

Algunas preguntas que pueden ayudar a elegir:

¿Qué grado de orden tendrá la información que se manejará?

Si la información va a estar casi ordenada y uno no quiere complicarse, un algoritmo sencillo como el ordenamiento burbuja será suficiente. Si por el contrario los datos van a estar muy desordenados, un algoritmo poderoso como Quicksort puede ser el más indicado. Y si no se puede hacer una presunción sobre el grado de orden de la información, lo mejor será elegir un algoritmo que se comporte de manera similar en cualquiera de estos dos casos extremos.

¿Qué cantidad de datos se va a manipular?

Si la cantidad es pequeña, no es necesario utilizar un algoritmo complejo, y es preferible uno de fácil implementación. Una cantidad muy grande puede hacer prohibitivo utilizar un algoritmo que requiera de mucha memoria adicional.

¿Qué tipo de datos se quiere ordenar?

Algunos algoritmos sólo funcionan con un tipo específico de datos (enteros, enteros positivos, etc.) y otros son generales, es decir, aplicables a cualquier tipo de dato.

¿Qué tamaño tienen los registros de la lista?

Algunos algoritmos realizan múltiples intercambios (burbuja, inserción). Si los registros son de gran tamaño estos intercambios son más lentos.

Capítulo 11

Estructuras y uniones

Al contrario que los arrays, las estructuras nos permiten agrupar varios datos, que mantengan algún tipo de relación, aunque sean de distinto tipo, permitiendo manipularlos todos juntos, usando un mismo identificador, o cada uno por separado.

Las estructuras son llamadas también muy a menudo registros, o en inglés *records*. Tienen muchos aspectos en común con los registros usados en bases de datos. Y siguiendo la misma analogía, cada objeto de una estructura se denomina a menudo campo, o field.

Sintaxis:

```
struct [<identificador>] {
   [<tipo> <nombre_objeto>[,<nombre_objeto>,...]];
} [<objeto_estructura>[,<objeto_estructura>,...];
```

El identificador de la estructura es un nombre opcional para referirse a la estructura.

Los objetos de estructura son objetos declarados del tipo de la estructura, y su inclusión también es opcional. Si bien, aún siendo ambos opcionales, al menos uno de estos elementos debe existir.

En el interior de una estructura, entre las llaves, se pueden definir todos los elementos que sean considerados necesarios, del mismo modo que se declaran los objetos.

Las estructuras pueden referenciarse completas, usando su nombre, como se hacen con los objetos que ya son conocidos, y también se puede acceder a los elementos definidos en el interior de la estructura, usando el operador de selección (.), un punto.

Una vez definida una estructura, es decir, si se ha especificado un nombre para ella, se puede usar igual que cualquier otro tipo de C++. Esto significa que se pueden declarar más objetos del tipo de estructura en cualquier parte del programa. Para ello se utiliza la forma normal de declaración de objetos, es decir:

```
[struct] <identificador> <objeto_estructura>
   [,<objeto_estructura>...];
```

En C++ la palabra struct es opcional en la declaración de objetos, al contrario de lo que sucede en C, en el que es obligatorio usarla.

Ejemplo:

```
struct Persona {
   char Nombre[65];
   char Direccion[65];
   int AnyoNacimiento;
} Fulanito;
```

Este ejemplo define la estructura Persona y declara a Fulanito como un objeto de ese tipo. Para acceder al nombre de Fulanito, por ejemplo para visualizarlo, usaremos la forma:

```
cout << Fulanito.Nombre;
```

Funciones en el interior de estructuras

C++, permite incluir funciones en el interior de las estructuras. Normalmente estas funciones tienen la misión de manipular los datos incluidos en la estructura, y su uso está muy relacionado con la programación orientada a objetos.

Aunque esta característica se usa casi exclusivamente con las clases, también puede usarse en las estructuras. De hecho, en C++, las diferencias entre estructuras y clases son muy tenues.

Dos funciones muy particulares son las de inicialización, o constructor, y el destructor.

El constructor es una función sin tipo de retorno y con el mismo nombre que la estructura. El destructor tiene la misma forma, salvo que el nombre va precedido el símbolo "~".

Nota: para aquellos que usen un teclado español, el símbolo "~" se obtiene pulsando las teclas del teclado numérico 1, 2, 6, mientras se mantiene pulsada la tecla ALT, ([ALT]+126). También mediante la combinación [Atl Gr]+[4] y un espacio (la tecla [4] de la zona de las letras, no del teclado numérico).

Ejemplo 1:

```
struct Punto {
   int x, y;
   Punto() {x = 0; y = 0;} // Constructor
} Punto1, Punto2;
```

Ejemplo 2:

```
struct Punto {
   int x, y;
   Punto(); // Declaración del constructor
} Punto1, Punto2;

// Definición del constructor, fuera de la estructura
Punto::Punto() {
   x = 0;
   y = 0;
}
```

Si no se utiliza un constructor, los valores de x e y para Punto1 y Punto2 estarían indeterminados, contendrían la "basura" que hubiese en la memoria asignada a estas estructuras durante la ejecución. Con las estructuras éste será el caso más habitual, ya que si necesitamos usar constructores para asignar valores iniciales, será mucho más lógico usar clases que estructuras.

Otro ejemplo:

```
#include <iostream>
using namespace std;

struct stPareja {
   int A, B;
   int LeeA() { return A;} // Devuelve el valor de A
   int LeeB() { return B;} // Devuelve el valor de B
   void GuardaA(int n) { A = n;} //Asigna nuevo valor a A
   void GuardaB(int n) { B = n;} // Asigna nuevo valor a B
} Par;

int main() {
   Par.GuardaA(15);
   Par.GuardaB(63);
   cout << Par.LeeA() << endl;
   cout << Par.LeeB() << endl;

   return 0;
}
```

En este ejemplo se puede ver cómo se define una estructura con dos campos enteros, y dos funciones para modificar y leer sus valores. El ejemplo es muy simple, pero las funciones de guardar valores se pueden elaborar para que no permitan determinados valores, o para que hagan algún tratamiento de los datos.

Inicialización de estructuras

De un modo parecido al que se inicializan los arrays, se pueden inicializar estructuras, tan sólo hay que tener cuidado con las estructuras anidadas. Por ejemplo:

```
struct A {
   int i;
   int j;
   int k;
};
struct B {
   int x;
   struct C {
      char c;
      char d;
   } y;
   int z;
};
A ejemploA = {10, 20, 30};
B ejemploB = {10, {'a', 'b'}, 20};
```

Cada nueva estructura anidada deberá inicializarse usando la pareja correspondiente de llaves "{}", tantas veces como sea necesario.

Uniones

Las uniones son un tipo especial de estructuras que permiten almacenar elementos de diferentes tipos en las mismas posiciones de memoria, aunque evidentemente, no en forma simultánea.

Sintaxis:

```
union [<identificador>] {
   [<tipo> <nombre_variable>[,<nombre_variable>,...]];
} [<variable_union>[,<variable_union>,...];
```

El identificador de la unión es un nombre opcional para referirse a la unión.

Las variables de unión son objetos declarados del tipo de la unión, y su inclusión también es opcional.

Sin embargo, como en el caso de las estructuras, al menos uno de estos elementos debe existir, aunque ambos sean opcionales.

En el interior de una unión, entre las llaves, se pueden definir todos los elementos necesarios, del mismo modo que se declaran los objetos. La particularidad es que cada elemento se almacenará comenzando en la misma posición de memoria.

Las uniones pueden referenciarse completas, usando su nombre, como con las estructuras, y también se puede acceder a los elementos en el interior de la unión usando el operador de selección (.), un punto.

También pueden declararse más objetos del tipo de la unión en cualquier parte del programa, de la siguiente forma:

```
[union] <identifiador_de_unión> <variable>[,<variable>...];
```

La palabra clave union es opcional en la declaración de objetos en C++. Aunque en C es obligatoria.

Ejemplo:

```
#include <iostream>
using namespace std;

union unEjemplo {
   int A;
   char B;
   double C;
} UnionEjemplo;

int main() {
   UnionEjemplo.A = 100;
   cout << UnionEjemplo.A << endl;
   UnionEjemplo.B = 'a';
   cout << UnionEjemplo.B << endl;
   UnionEjemplo.C = 10.32;
   cout << UnionEjemplo.C << endl;
   cout << &UnionEjemplo.A << endl;
   cout << (void*)&UnionEjemplo.B << endl;
   cout << &UnionEjemplo.C << endl;
   cout << sizeof(unEjemplo) << endl;
   cout << sizeof(UnionEjemplo.A) << endl;
   cout << sizeof(UnionEjemplo.B) << endl;
   cout << sizeof(UnionEjemplo.C) << endl;

   return 0;
}
```

Inicialización de uniones

Las uniones solo pueden ser inicializadas en su declaración mediante su primer miembro.

Por ejemplo, en la primera unión:

```
union unEjemplo {
   int A;
   char B;
   double C;
} UnionEjemplo;
```

Se puede iniciar objetos de este tipo asignando un entero:

```
unEjemplo x = {10}; // int
unEjemplo y = {'a'}; // char
unEjemplo z = {10.02}; // double
```

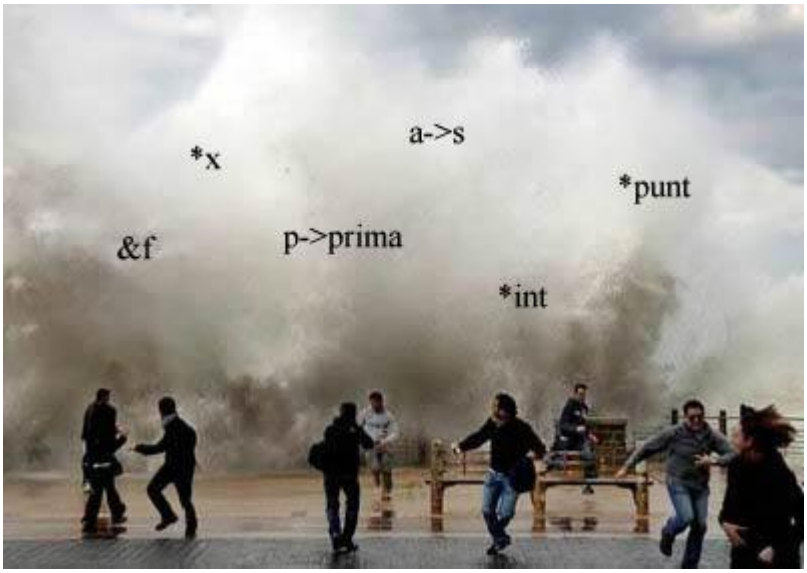
Al utilizar un carácter, como en el caso de 'y', se hará una conversión de tipo a int de forma automática, y se asignará el valor correcto. En el caso de 'z', se produce un aviso, por democión automática.

Al igual que pasa con otros tipos agregados, como arrays y estructuras, hay que usar llaves para incluir una lista de valores iniciales. En el caso de la unión, esa lista tendrá un único elemento, ya que todos los miembros comparten la misma zona de memoria, y sólo está permitido usar el primero para las inicializaciones.

Capítulo 12

Punteros

Estimado lector, no salga corriendo!. Aunque es un tema que suele asustar a los estudiantes de C++, no es algo tan terrible como se cuenta!.



"El ataque de los punteros"

Los punteros proporcionan la mayor parte de la potencia al C++, y marcan la principal diferencia con otros lenguajes de programación.

Fuera del ámbito de la programación. Se utilizan punteros para señalar cosas sobre las que se quiere llamar la atención, como marcar puntos en un mapa o detalles en una presentación en pantalla. A menudo, se utiliza el dedo índice para señalar direcciones o lugares sobre los que se está hablando o explicando algo. Cuando un dedo no es suficiente, se utiliza un puntero (o apuntador). Antiguamente esos punteros eran una vara de madera, pero actualmente se usan punteros laser, aunque la idea es la misma. Un puntero también es el símbolo que representa la posición del ratón en una pantalla gráfica. Estos punteros también se usan para señalar objetos: enlaces, opciones de menú, botones, etc. Un puntero sirve, pues, para apuntar a los objetos a los que se está uno refiriendo.

En C++ un puntero es exactamente lo mismo. Los punteros en C++ sirven para señalar objetos, y también para manipularlos.

Para entender qué es un puntero es necesario conocer cómo se almacenan los datos en un ordenador.

La memoria de un ordenador está compuesta por unidades básicas llamadas bits. Cada bit sólo puede tomar dos valores, normalmente denominados alto y bajo, ó 1 y 0. Pero trabajar con bits no es práctico, y por eso se agrupan.

Cada grupo de 8 bits forma un byte u octeto. En realidad el microprocesador, y por lo tanto el programa, sólo puede manejar directamente bytes o grupos de dos o cuatro bytes. Para acceder a los bits hay que acceder antes a los bytes.

Cada byte de la memoria de un ordenador tiene una dirección, llamada dirección de memoria.

Los microprocesadores trabajan con una unidad básica de información, a la que se denomina palabra (en inglés word). Dependiendo del tipo de microprocesador una palabra puede estar compuesta por uno, dos, cuatro, ocho o dieciséis bytes. Se habla indistintamente de direcciones de memoria, aunque las palabras sean de distinta longitud. Cada dirección de memoria contiene siempre un byte. Lo que sucederá cuando las palabras sean, por ejemplo, de 32 bits es que se acceden a posiciones de memoria que son múltiplos de 4.

Por otra parte, la mayor parte de los objetos que se utilizan en los programas no caben en una dirección de memoria. La solución utilizada para manejar objetos que ocupen más de un byte es usar posiciones de memoria correlativas. De este modo, la dirección de un objeto es la dirección de memoria de la primera posición que contiene ese objeto.

Dicho de otro modo, si para almacenar un objeto se precisan cuatro bytes, y la dirección de memoria de la primera posición es n, el objeto ocupará las posiciones desde n a n+3, y la dirección del objeto será, también, n.

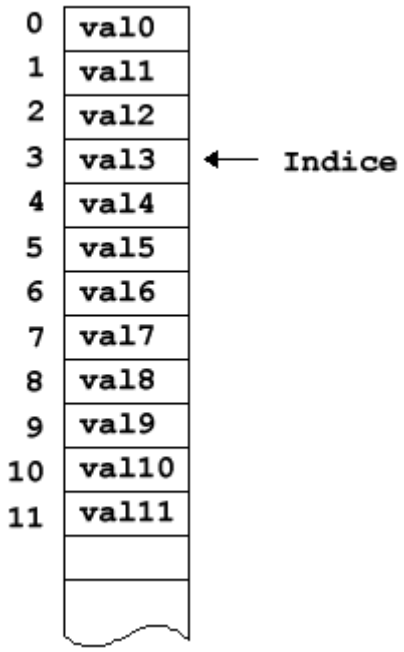
Un puntero es un tipo especial de objeto que contiene, ni más ni menos que, la dirección de memoria de un objeto.

Por supuesto, almacenada a partir de esa dirección de memoria puede haber cualquier clase de objeto: un char, un int, un float, un array, una estructura, una función u otro puntero. El programador es el responsable de decidir ese contenido, al declarar el puntero.

De hecho, existen tantos tipos diferentes de punteros como tipos de objetos puedan ser referenciados mediante punteros. Si se tiene esto en cuenta, los punteros que apunten a tipos de objetos distintos, serán tipos diferentes. Por ejemplo, no es permitido asignar a un puntero a char el valor de un puntero a int.

Como funciona un puntero

Se considera la memoria del ordenador como un gran array, de modo que es posible acceder a cada celda de memoria a través de un índice en donde la primera posición del array es la celda[0].



Indice <-> puntero

Si se utiliza una variable para almacenar el índice, por ejemplo, indice=0, entonces celda[0] == celda[indice]. Finalmente, si se prescinde de la notación de los arrays, el índice se comporta exactamente igual que un puntero.

El puntero indice podría tener por ejemplo, el valor 3, en ese caso, el valor apuntado por indice tendría el valor 'val3'.

Las celdas de memoria tienen una existencia física, es decir son algo real y existirán siempre, independientemente del valor del puntero. Existirán incluso si no existe el puntero.

De forma recíproca, la existencia o no existencia de un puntero no implica la existencia o la inexistencia del objeto. De la misma forma que el hecho de no señalar a un árbol, no implica la inexistencia del árbol. Algo más oscuro es se tiene un puntero para árboles, que no esté señalando a un árbol, un puntero de ese tipo no tendría uso si uno está en medio del mar: tener ese puntero no crea árboles de forma automática cuando se señala con él, por tanto, es un puntero, no una varita mágica.

Del mismo modo, el valor de la dirección que contiene un puntero no implica que esa dirección sea válida, en el sentido de que no tiene por qué contener la dirección de un objeto del tipo especificado por el puntero.

Suponiendo que se cuenta con un mapa en la pared, y también que existen diferentes tipos de punteros láser para señalar diferentes tipos de puntos en el mapa. Se crea un puntero para señalar ciudades. En el momento de crearlo (o encenderlo), el puntero señalará a cualquier sitio, podría señalar incluso a un punto fuera del mapa. En general, se dá por sentado que una vez creado, el puntero no tiene por qué apuntar a una ciudad, y aunque apunte al mapa, podría estar señalando a un mar o a un río.

Con los punteros en C++ ocurre lo mismo. El valor inicial del puntero, cuando se declara, podría estar fuera del mapa, es decir, contener direcciones de memoria que no existen. Pero, incluso señalando a un punto de la memoria, es muy probable que no señale a un objeto del tipo adecuado. Es necesario considerar esto como el caso más probable, y no usar jamás un puntero que no haya sido inicializado correctamente.

Dentro del array de celdas de memoria existirán zonas que contendrán programas y datos, tanto del usuario como del propio sistema operativo o de otros programas, el sistema operativo se encarga de gestionar esa memoria, prohibiendo o protegiendo determinadas zonas.

Pero el propio puntero, como objeto que es, también se almacenará en memoria, y por lo tanto, también tendrá una dirección de memoria. Cuando se declara un puntero se está reservando la memoria necesaria para almacenarlo, aunque, como pasa con el resto de los objetos, el contenido de esa memoria contendrá basura.

En principio, se debe asignar a un puntero, o bien la dirección de un objeto existente, o bien la de uno creado explícitamente durante la ejecución del programa o un valor conocido que indique que no señala a ningún objeto, es decir el valor 0. El sistema operativo, cuanto más avanzado es, mejor suele controlar la memoria. Ese control se traduce en impedir el acceso a determinadas direcciones reservadas por el sistema.

Declaración de punteros

Los punteros se declaran igual que el resto de los objetos, pero precediendo el identificador con un asterisco (*).

Sintaxis:

```
<tipo> *<identificador>;
```

Ejemplos:

```
int *pEntero;
char *pCaracter;
struct stPunto *pPunto;
```

Los punteros sólo pueden apuntar a objetos de un tipo determinado, en el ejemplo, pEntero sólo puede apuntar a un objeto de tipo int.

La forma:

```
<tipo>* <identificador>;
```

con el (*) junto al tipo, en lugar de junto al identificador del objeto, también está permitida. De hecho, también es legal la forma:

```
<tipo> * <identificador>;
```

Por lo tanto:

```
int *pEntero;
```

equivale a:

```
int* pEntero;
```

Otro detalle importante es que, aunque las tres formas de situar el asterisco en la declaración son equivalentes, algunas de ellas pueden inducir a error, sobre todo si se declaran varios objetos en la misma línea:

```
int* x, y;
```

En este caso, `x` es un puntero a int, pero `y` no es más que un objeto de tipo int. Colocar el asterisco junto al tipo puede que indique más claramente que se está declarando un puntero, pero hay que tener en cuenta que sólo afecta al primer objeto declarado, si se quiere declarar ambos objetos como punteros a int se debe de hacerlo de otro modo:

```
int* x, *y;
// O:
int *x, *y;
// O:
int* x;
int* y;
```

Obtener punteros a objetos

Los punteros apuntan a objetos, por lo tanto, lo primero que se debe saber hacer con los punteros es asignarles direcciones de memoria válidas de objetos.

Para averiguar la dirección de memoria de cualquier objeto se utiliza el operador de dirección (&), que se lee como "dirección de".

Por supuesto, los tipos tienen que ser "compatibles", no es permitido almacenar la dirección de un objeto de tipo char en un puntero de tipo int.

Por ejemplo:

```
int A;
int *pA;

pA = &A;
```

Según este ejemplo, `pA` es un puntero a int que apunta a la dirección donde se almacena el valor del entero `A`.

Objeto apuntado por un puntero

La operación contraria es obtener el objeto referenciado por un puntero, con el fin de manipularlo, ya sea modificando su valor u obteniendo el valor actual.

Para manipular el objeto apuntado por un puntero se utiliza el operador de indirección, que es un asterisco (*).

En C++ es muy habitual que el mismo símbolo se use para varias cosas diferentes, este es el caso del asterisco, que se usa como operador de multiplicación, para la declaración de punteros y como operador de indirección.

Como operador de indirección sólo está permitido usarlo con punteros, y se leer "objeto apuntado por".

Por ejemplo:

```
int *pEntero;
int x = 10;
int y;

pEntero = &y;
*pEntero = x;
```

En la última línea se asigna al objeto apuntado por `pEntero` el valor del objeto x. Como `pEntero` apunta al objeto y, esta sentencia equivale (según la secuencia del programa), a asignar a `y` el valor de `x`.

Diferencia entre punteros y otros objetos

Hay que tener muy claro, en el ejemplo anterior, que `pEntero` es un objeto del tipo "puntero a int", pero que `*pEntero` NO es un objeto de tipo int, sino una expresión.

Como pasa con todos los objetos en C++, cuando se declaran, sólo se reserva espacio para almacenarlos, pero no se asigna ningún valor inicial, (recordar que el puntero para árboles no crea un árbol cada vez que se señala con él). El contenido del objeto permanecerá sin cambios, de modo que el valor inicial del puntero será aleatorio e indeterminado. Es de suponer que contiene una dirección no válida.

Si `pEntero` apunta a un objeto de tipo int, `*pEntero` será el contenido de ese objeto, pero no hay que olvidar que `*pEntero` es un operador aplicado a un objeto de tipo "puntero a int". Es decir, `*pEntero` es una expresión, no un objeto.

Declarar un puntero no creará un objeto del tipo al que apunta. Por ejemplo: int `*pEntero`; no crea un objeto de tipo int en memoria. Lo que crea es un objeto que puede contener la dirección de memoria de un entero.

Se puede decir que existe físicamente un objeto `pEntero`, y también que ese objeto puede (aunque esto no es siempre cierto) contener la dirección de un objeto de tipo int.

Como todos los objetos, los punteros también contienen "basura" cuando son declarados. Es costumbre dar valores iniciales nulos a los punteros que no apuntan a ningún sitio concreto:

```
int *pEntero = 0; // También se puede asignar el valor NULL
char *pCaracter = 0;
```

NULL es una constante, que está definida como cero en varios ficheros de cabecera, como "cstdio" o "iostream", y normalmente vale 0L. Sin embargo, hay muchos textos que recomiendan usar el valor 0 para asignar a punteros nulos, al menos en C++.

Repasando punteros:

Un puntero es una variable que contiene la dirección de memoria de otra variable.

Los punteros permiten código más compacto y eficiente; utilizándolos en forma ordenada dan gran flexibilidad a la programación.

La dirección de memoria de una variable se obtiene con el operador unario &.

El operador unario * permite la desreferencia de un variable puntero; es decir, permite el acceso a lo apuntado por un puntero.

Dado el ejemplo:

```
int x = 1, y = 2;
int *ip; /* ip es un puntero a int */
ip = &x; /* ip apunta a x */
y = *ip; /* a y se le asigna lo apuntado por ip */
*ip = *ip + 3; /* incrementa lo apuntado por ip, x */
ip = NULL; /* ip apunta a direcc. especial (nada) */
```

La sintaxis de la declaración de un puntero imita a las expresiones en que la variable puede utilizarse; cada puntero apunta a un tipo específico de datos (con la excepción del puntero genérico void).

La dirección especial NULL (o cero) indica que un puntero apunta a "nada" y es usada como centinela para establecer el fin de estructuras autoreferenciadas. Además, esta es retornada por la función de asignación de memoria, malloc, en el caso de no haber suficiente memoria. El operador new, en cambio, aborta el programa cuando no tiene más memoria que dar.

Problemas a evitar en el uso de punteros

El uso descuidado de punteros lleva a programas incomprensibles y particularmente a problemas difíciles de encontrar y reparar.

Ausencia de inicialización de punteros Un problema común es el uso de punteros no inicializados. Mientras las variables externas (a funciones) o estáticas son inicializadas en cero por omisión, las variables locales (pertenecientes a funciones) no son inicializadas automáticamente. Por lo tanto punteros definidos dentro de funciones que no sean inicializados pueden contener cualquier valor ("basura").

Es un error desreferenciar un puntero con valor NULL,

```
int *ip = NULL;
cout << *ip << endl;
```

provoca la excepción de violación de acceso de memoria (segmentation fault) y detiene el programa.

También es erróneo usar un puntero no inicializado,

```
int *ip;
cout << *ip << endl;
```

Los compiladores en general informan (warning) que se está utilizando una variable no inicializada.

Su ejecución tiene un comportamiento aleatorio, dependiendo del valor que casualmente exista en el área de memoria asignada a la variable puntero. Puede llegar a funcionar incorrectamente si la información que hay en memoria casualmente es un entero válido; pero probablemente provocará la detención del programa. En estructuras de datos dinámicas la ejecución del programa en estas condiciones es errática, luego de ejecutarse incorrectamente por cierto tiempo se detiene ("cuelga") el programa.

Potencialmente mas grave es modificar un área de memoria que no fue definida explícitamente,

```
int *ip;
*ip = 10;
```

No solo modifica un espacio de memoria que no le fue asignado; sino que, al no estar inicializado el puntero, se modifica el espacio de memoria al que casualmente direccione el puntero. La declaración int *ip solo asigna memoria para la variable puntero, no para los datos apuntados por éste. En general (lo cual no quiere decir que ocurra siempre) provoca la excepción de violación de acceso de memoria (segmentation fault) y detiene el programa.

Una forma adecuada de hacerlo es:

```
int *ip;
ip = new int; //(1)
*ip = 10;
```

Donde la función new establece memoria para el entero y la asignación inicializa el puntero. En caso de querer utilizar estrictamente el lenguaje C (el operador new es propio de C++), la sentencia (1) pasaría a ser la siguiente:

```
ip = (int*) malloc (sizeof (int)); //(1)
```

La asignación de punteros se puede realizar adecuadamente al establecer su valor inicial en NULL, y luego asignar memoria previo a su desreferenciamiento,

- asignar un puntero a otro que a sido adecuadamente inicializado, o

- usar las funciones new o malloc para reservar memoria.

Ejercicio:

Escribir un programa en lenguaje C que: Lea un número n y lo acepte sólo si es entero y positivo, en caso contrario, solicite nuevamente el número n, hasta obtener uno entero y positivo, y los n×n elementos de la matriz cuadrada mat. Imprima luego el número n, los elementos de la matriz mat y dos números s y t, obtenidos, respectivamente, como la suma de los elementos de mat cuya suma de índices es par y como la suma de los elementos de mat cuya suma de índices es impar.

```
#include <iostream>
using namespace std;
void leer(float *A,int f,int c);
void print(float *A,int f,int c);
int ent (void);
main()
{
int i, j, n;
float s=0, t=0;
cout<<"\n\t\tDigite el orden de la matriz: ";
n=ent();
float mat[n][n];
leer(mat[0],n,n);
for ( i = 0; i < n; i++ )//entrada de la matriz y suma
{
    for ( j = 0; j < n; j++ )
    {
    if ( (i + j)%2 == 0 )s += mat[i][j];
    else{ t += mat[i][j]; }
    }
}
cout<<"\n\t\tOrden de la matriz:
"<<n<<"\t"<<sizeof(mat)<<"\t"<<sizeof(mat[0])<<"\t"<<sizeof(mat[
0][0]);
print(mat[0],n,n);
cout<<"\n\n\t\tSuma de los elementos cuya suma de índices es par
: "<<s;
cout<<"\n\t\tSuma de los elementos cuya suma de índices es
impar: "<<t<<endl;
system ("PAUSE");
}
```

```
void print(float *A,int f,int c)
{
    for ( int i = 0; i < f; i++ )
    {
        cout<<"\n\t|\t";
        for(int j=0;j<c;j++) cout<<*(A+c*i+j)<<"\t";
        cout<<"|";
    }
}

void leer(float *A,int f,int c)
{
    cout<<"\tIntroduzca los elementos de la matriz:\n";
    for ( int i = 0; i < f; i++ )
    {

        for(int j=0;j<c;j++)
        {
            cout<<"["<<i<<"]["<<j<<"]=";
            cin>>*(A+c*i+j);
        }
    }
}

int ent (void)
{
    float x;
    cin>>x;
    while(x<=0 || x!=int(x))
    {
        cout<<"\n El valor debe ser entero y positivo introduzca
de nuevo:\n";
        cin>>x;
    }
    return int(x);
}
```

Apéndice

Ejercicios

1. Busqueda secuencial en un vector

```
#include<iostream>
using namespace std;
main()
{     int vect[100], i, a;
      srand( time(NULL)  );
      for ( i = 0; i < 100; i++ )
            vect[i] = rand()%1000 + 1;
      for ( i = 0; i < 100; i++ )
      {    if ( i%10 == 0 ) cout << "\n";
          cout << vect[i] << ",  ";   }
   do {         cout << "\nDigite el numero a buscar ";
           cin >> a;
           if ( a <= 0 ) break;
           for ( i = 0; i < 100; i++ )
           {                if ( a == vect[i] ) break;        }
         if ( i < 100 )
         {           cout << "\n\tEl numero " << a;
               cout << " ocupa la posicion " << i + 1;
               cout << " en el vector";       }
      else
      {            cout << "\n\tEl numero " << a;
               cout << " no se encuentra en el vector";      }
      }
 while( a );}
```

2. Determinacion del intervalo de tiempo entre la hora inicial y final de una actividad

```
#include<iostream>
using namespace std;
int validah ( int h );
int validams ( int m );
int validaint ( float f );
int main()
{    int hi, mi, si, ti, hf, mf, sf, tf, dh, dm, ds, dt, error;
    float aux;
           error = 0;
           while ( error == 0 )
           {     cout << "\nDigite la hora inicial   : ";
                 cin >> aux;
                 error = validaint( aux );
                           if ( error == 1 )
```

```
                                {                    hi = (int)aux;
                                                     error = validah ( hi
);          }     }
            error = 0;
            while ( error == 0 )
            {     cout << "\nDigite el minuto inicial : ";
                  cin >> aux;
                  error = validaint( aux );
                              if ( error == 1 )
                              {      mi = (int)aux;
                                     error = validams ( mi );
}     }
            error = 0;
            while ( error == 0 )
            {     cout << "\nDigite el segundo inicial: ";
                  cin >> aux;
                  error = validaint( aux );
                  if ( error == 1 )
                  {          si = (int)aux;
                            error = validams ( si );            }
      }
            ti = si + mi*60 + hi*3600;
            error = 0;
            while ( error == 0 )
            {     cout << "\nDigite la hora final   : ";
                  cin >> aux;
                  error = validaint( aux );
                  if ( error == 1 )
                  {                hf = (int)aux;
                              error = validah ( hf );
      }     }
            error = 0;
            while ( error == 0 )
            {     cout << "\nDigite el minuto final : ";
                  cin >> aux;
                  error = validaint( aux );
                  if ( error == 1 )
                  {              mf = (int)aux;
                              error = validams ( mf );        }
      }
            error = 0;
            while ( error == 0 )
            {     cout << "\nDigite el segundo final: ";
                  cin >> aux;
                  error = validaint( aux );
                  if ( error == 1 )
                  {                        sf = (int)aux;
                                         error = validams ( sf );
      }           }
            tf = sf + mf*60 + hf*3600;
            if ( tf <= ti )
            {      cout << "\nEl tiempo final no es mayor que el
inicial\n";
```

```
                system ( "pause" );
                return 0;       }
            dt = tf - ti;
            dh = dt/3600;
            dt = dt%3600;
            dm = dt/60;
            ds = dt%60;
            cout << "\nLa diferencia de tiempo es ";
            cout << dh << " h " << dm << " min " << ds << " s";
    cout << "\n";
    system ( "pause" );}
int validah ( int h )
{           if ( h < 0 || h > 23 ) return 0;    // con error
            return 1;                           // valor correcto
}
int validams ( int m )
{           if ( m < 0 || m > 59 ) return 0;
            return 1; }
int validaint ( float f )
{           if ( f != (int)f ) return 0;
            return 1;}
```

3. Calculo del factorial de un numero utilizando recursividad

```
#include<iostream>
using namespace std;
int factorial( int n );
int main()
{       int n;
        do {
             cout << "\nDigite un numero entero no negativo ";
             cin >> n;     }  while ( n < 0 );
    cout << "\n\t" << n << "! = " << factorial( n ) << "\n\n\t";
    system ( "pause" ); }
int factorial( int n )
{       if ( n <= 1 ) return 1;
        cout << "\n\t n = " << n;
        return n*factorial( n - 1 );}
```

4. Numeros de Fibonacci

```
#include<iostream>
using namespace std;
int main()
{      int i, j, k, n;
       do {   cout << "\nDigite el numero de terminos de la serie
de Fibonacci a generar ";
            cin >> n;     }  while ( n < 0 );
    i = 0, j = 1;
       cout << "\n\t fib = { " << i << ", " << j;
    for ( k = 1; k < n; k++ )
    {        j = i + j;
             i = j - i;
        cout << "; " << j;   }
       cout << " }\n\n";
    system ( "pause" );}
```

5. Numeros de Fibonacci. Utilizando recursividad

```
#include<iostream>
using namespace std;
int fibonacci( int n );
int main()
{      int i, n;
       do {   cout << "\nDigite un numero entero no negativo ";
            cin >> n;     }  while ( n < 0 );
       cout << "\n\t fib = { " << fibonacci( 0 );
    for ( i = 1; i < n; i++ )
        cout << "; " << fibonacci( i );
       cout << " }\n\n";
    system ( "pause" );}
int fibonacci( int n )
{      if ( n <= 1 ) return n;
       return ( fibonacci( n - 1 ) + fibonacci( n - 2 ) );}
```

6. Vector como parametro de entrada de una funcion (Paso de un argumento por referencia)

```
#include<iostream>
using namespace std;
void modifVect( float vect[], int n );
main()
{          int i, n;
           float nf;
```

```
        cout << "\nDigite un numero entero y positivo ";
          cin >> nf;
          while ( nf <= 0 || nf != (int)nf )
          {     cout << "\nNumero invalido, digite nuevamente ";
                cin >> nf;      }
    n = (int)nf;
    float v[n];
          for ( i = 0; i < n; i++ )
          {          cout << "\nDigite v[" << i << "] ";
                     cin >> v[i];          }
          cout << "\nVector leido ";
          for ( i = 0; i < n; i++ )
                     cout << v[i] << ", ";
          modifVect( v, n );
          cout << "\nNuevo vector ";
          for ( i = 0; i < n; i++ )
                     cout << v[i] << ", ";
    cout << "\n";
    system( "pause" );}
void modifVect( float vect[], int n )
{         int i;
          for ( i = 0; i < n; i++ )
                     vect[i] += 2;            // Aumenta las
componentes en dos}
```

7. Se dispone de dos vectores, maestro y esclavo, de componentes enteras y positivas e igual numero de elementos. Se deben imprimir en dos columnas adyacentes. Se ordena el vector maestro, pero siempre que un elemento de maestro se mueva, el elemento correspondiente de escalvo debe moverse tambien; es decir, cualquier accion hecha con maestro[i] debe hacerse a esclavo[i]. Despues de ralizar la ordenación se imprimen de nuevo los vectores. Escribir un programa que realice esta tarea. Observacion: Utilizar como algoritmo de ordenacion, el metodo de insercion .

```
#include<iostream>
using namespace std;
int main()
{   int i, j, n, auxMaes, auxEscl, randMax = 100;
/* Variables del programa.
     Entrada:        n      : numero componentes de los
vectores.
                maestro: vector de n componentes (se genera con
numeros pseudoaleatorios)
                esclavo: vector de n compoentens (se genera con
numeros naturales en secuencia)
   Salida:        maestro: vector ordenado en secuencia
ascendente
            esclavo: vector indicando las posiciones iniciales
de la componentes de maestro  */
     do {  cout << "\nDigite un numero entero ";
           cin >> n;
```

```
      }     while ( n < 2 );
   int maestro[n], esclavo[n];
// Generacion de datos
   srand( time( NULL ) );
   for ( i = 0; i < n; i++ )
      {     maestro[i] = rand()%randMax + 1;
            esclavo[i] = i + 1;   }
// Impresion de los vectores generados
   cout << "\tmaestro  esclavo";
   for ( i = 0; i < n; i++ )
   { cout << "\n\t   " << maestro[i] << "\t    " << esclavo[i];
}
   cout << "\n";
      system ( "pause" );
// Ordenacion de maestro y movimiento de esclavo
      for ( i = 1; i < n; i++ )
      {     auxMaes = maestro[i];
            auxEscl = esclavo[i];
            j = i - 1;
            while ( j >= 0 && auxMaes < maestro[j] )
            {     maestro[j+1] = maestro[j];
                  esclavo[j+1] = esclavo[j];
                  j--;  }
            maestro[j+1] = auxMaes;
            esclavo[j+1] = auxEscl;         }
// Impresion de los vectores ordenados
   cout << "\tmaestro  esclavo";
   for ( i = 0; i < n; i++ )
   { cout << "\n\t   " << maestro[i] << "\t    " << esclavo[i];
}
   cout << "\n";
      system ( "pause" );}
```

8. Uso de un vector de componentes enteras y positivas

```
#include<iostream>
using namespace std;
int indice( int *cont );
main()
{                 int i, n, menor, contMen, var;
                  var = 0;
                  n = indice( &var );
                  cout << "\nParametro de entrada " << var;
                  int vect[n], posMen[n];
                  var = 1;
                  for ( i = 0; i < n; i++ )
                            vect[i] = indice( &var );
                  cout << "\nValor del parametro de entrada " <<
var;
                  cout << "\nvect = { " << vect[0];
                  for ( i = 1; i < n; i++ )
                            cout << ", " << vect[i];
```

```
            cout << " }";
            contMen = 0;
            menor = vect[0];
            posMen[contMen] = 0;
            for ( i = 1; i < n; i++ )
            {        if ( vect[i] == menor )
                          {      contMen++;
                                  posMen[contMen] = i;
      }
                          else
                          {     if ( vect[i] < menor )
                                     {      menor = vect[i];
                                            contMen = 0;
                                            posMen[contMen] =
i;}   }     }
            cout << "\n\nLa menor componente es " << menor;
            cout << " y ocupa la(s) posicion(es) { " <<
posMen[0] + 1;
            for ( i = 1; i <= contMen; i++ )
                     cout << ", " << posMen[i] + 1;
                cout << " }";
      cout << "\n";
      system( "pause" );}
int indice( int *cont )
{          float nf;
           if ( *cont )
                 cout << "\nDigite un numero entero y positivo
";
           else
                 cout << "\nDigite el numero de componentes del
vector ";
           cin >> nf;
           while ( nf <= 0 || nf != (int)nf )
           {     cout << "\nNumero invalido, digite nuevamente ";
                 cin >> nf;       }
           *cont = 10;
           return (int)nf; }
```

9. Ordenamiento por el metodo de la burbuja

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int leer( void );
main()
{     int i, j, n, ls, lg, aux, max = 100;  // max es el mayor
numero generado al azar
// Lectura del numero de componentes y carga del vector
      n = leer( );        int vect[n];       srand( time(NULL)  );
//    cout << "Rand_max = " << RAND_MAX;
      for ( i = 0; i < n; i++ )
         vect[i] = rand()%max + 1;
```

```
// Impresion de las componentes del vector
      cout << "\nNro. de componentes del vector " << n;
      cout << "\n\nLas componentes del vector son";
      for ( i = 0; i < n; i++ )
      {  if ( i%10 == 0 ) cout << "\n";
            cout << vect[i] << ", " ;  }
// Ordenamiento de las componentes del vector (Metodo de la
burbuja)
   ls = n;    lg = 1;
   for ( i = 0; ( i < n - 1 && lg == 1 ); i++ )
   { lg = 0;              ls--;
        for ( j = 0; j < ls; j++ )
           {     if ( vect[j] > vect[j+1] )
                 {    lg = 1;
                  aux = vect[j];
                  vect[j] = vect[j+1];
                  vect[j+1] = aux;    }  }  }
// Impresion de la componentes en secuencia ascendente
   cout << "\n\n";
      cout << "\nLas componentes ordenadas del vector son";
      for ( i = 0; i < n; i++ )
      {  if ( i%10 == 0 ) cout << "\n";
            cout << vect[i] << ", " ;  }
   cout << "\n";
   system( "pause" );  }
int leer( void )
{     float nf;
      cout << "\nDigite el numero de componentes del vector ";

      cin >> nf;
      while ( nf <= 0 || nf != (int)nf )
      {     cout << "\nNumero invalido, digite nuevamente ";
            cin >> nf; }
      return (int)nf;  }
```

10. Ordenar, en secuencia ascendente, las componentes de un vector cuyos elementos son numeros enteros y positivos menores que max, generados al azar. Utilizando el metodo de ordenacion por insercion.

```
#include<iostream>
void impresion( int *v, int li, int ls );
using namespace std;
int main()
{   int i, j, n, aux, max = 100;
// Lectura de datos.
   do {         cout << "Digite el numero de componentes del
vector ";
      cin >> n;
   } while ( n < 2 );
   int vect[n];
// Generacion de datos
```

```
   srand( time( NULL ) );
   for ( i = 0; i < n; i++ ) vect[i] = rand()%max + 1;
   cout << "\n\tDatos:";
   impresion( vect, 0, n - 1 );
// Ordenacion.
   for ( i = 1; i < n; i++ )
   {  aux = vect[i];
      for ( j = i - 1; (j >= 0 ) && (aux < vect[j]); j-- )
            vect[j + 1] = vect[j];
      vect[j + 1] = aux;   }
// Impresion de resultados.
   impresion( vect, 0, n - 1 );      cout << " }\n";}
void impresion( int *v, int li, int ls )
{  int i;
// Impresion de resultados.
   cout << "\tli = " << li << "; ls = " << ls << "\n";
   cout << "  v = { " << v[li];
   for ( i = li + 1; i <= ls; i++ )
   {    if ( i - li > 0 && (i - li )%15 == 0 ) cout << ",\n\t"
<< v[i];
           else cout << ", " << v[i];  }
   cout << " }\n\n";
   system ( "pause" );  }
```

11. Ordenamiento rapido (quicksort) - Ideado por Hoare -

```
#include<iostream>
using namespace std;
void quicksort( int *v, int izq, int der );
void impresion( int *v, int li, int ls );
int n;
int main()
{  int i, max = 100;   // max el el mayor numero al azar a ser
generado
// Lectura de datos.
   do {
      cout << "Digite el numero de componentes del vector ";
      cin >> n;
   }  while ( n < 2 );
   int vect[n];
   srand( time( NULL ) );
   for ( i = 0; i < n; i++ ) vect[i] = rand()%max + 1;
   cout << "\n\tDatos:";
   impresion( vect, 0, n - 1 );
// Ordenacion.
   quicksort ( vect, 0, n - 1 );
   cout << "\n\tDatos ordenados:";
   impresion( vect, 0, n - 1 );
}
void quicksort ( int *v, int izq, int der )
{   int i, j, aux1, aux2;
```

```
// Eleccion del pivote
   aux1 = v[(izq + der)/2];
   i = izq;                    j = der;
// Particion en dos subconjuntos (menores que el pivot y mayores
que el mismo)
   do  {       while ( i < der && v[i] < aux1 ) i++;
       while ( j > izq && aux1 < v[j] ) j--;
       if ( i <= j )
       {  aux2 = v[i];
              v[i] = v[j];       v[j] = aux2;
          i++;                j--;   }
   }  while( i <= j );
   if ( izq < j ) quicksort ( v, izq, j );
   if ( i < der ) quicksort ( v, i, der );}
void impresion( int *v, int li, int ls )
{  int i;
// Impresion de resultados.
   cout << "\tli = " << li << "; ls = " << ls << "\n";
   cout << "v = { " << v[0];
   for ( i = 1; i < n; i++ )
      cout << ", " << v[i];
   cout << " }\n\n";
   system ( "pause" );     }
```

12. Ordenamiento por seleccion.

```
#include<iostream>
#include<stdlib.h>
using namespace std;
int leer( void );
main()
{    int i, j, k, n, aux, max = 100;     // max es el mayor
numero generado al azar
// Lectura del numero de componentes y carga del vector
     n = leer( );
     int vect[n];
     srand( time(NULL)  );
//   cout << "Rand_max = " << RAND_MAX;
     for ( i = 0; i < n; i++ )
        vect[i] = rand()%max + 1;
// Impresion de las componentes del vector
     cout << "\nNro. de componentes del vector " << n;
     cout << "\n\nLas componentes del vector son";
     for ( i = 0; i < n; i++ )
     {  if ( i%10 == 0 ) cout << "\n";
           cout << vect[i] << ", " ;  }
// Ordenamiento de las componentes del vector (Metodo de
seleccion)
   for ( i = 0; i < n - 1; i++ )
   {  k = i;
        for ( j = i + 1; j < n; j++ )
```

```
                if ( vect[j] < vect[k] ) k = j;
            aux = vect[i];
            vect[i] = vect[k];          vect[k] = aux;        }
// Impresion de la componentes en secuencia ascendente
   cout << "\n\n";
      cout << "\nLas componentes ordenadas del vector son";
      for ( i = 0; i < n; i++ )
      {  if ( i%10 == 0 ) cout << "\n";
            cout << vect[i] << ", " ;  }
   cout << "\n";
   system( "pause" );}
int leer( void )
{     float nf;
      cout << "\nDigite el numero de componentes del vector ";

      cin >> nf;
      while ( nf <= 0 || nf != (int)nf )
      {     cout << "\nNumero invalido, digite nuevamente ";
            cin >> nf; }
      return (int)nf;}
```

13. Ordenar en secuencia ascendente las componentes de un vector cuyos elementos son numeros enteros y positivos menores que max, generados al azar.

```
#include<iostream>
void impresion( int *v, int li, int ls );
using namespace std;
int main()
{
   int i, j, aux, paso, n, max = 100;
// Lectura de datos.
   do
   {
      cout << "Digite el numero de componentes del vector ";
      cin >> n;
   } while ( n < 2 );
   int vect[n];
// Generacion de datos
   srand( time( NULL ) );
   for ( i = 0; i < n; i++ ) vect[i] = rand()%max + 1;
   cout << "\n\tDatos:";
   impresion( vect, 0, n - 1 );
// Ordenacion.
   paso = n/2;
   while ( paso > 0 )
   {
       for ( i = paso; i < n; i++ )
       {
           aux = vect[i];
           for ( j = i - paso; (j >= 0 ) && (aux < vect[j]); j -
= paso )
```

```
                vect[j + paso] = vect[j];
            vect[j + paso] = aux;
        }
        paso /= 2;
    }
    cout << "\n\tDatos ordenados:";
    impresion( vect, 0, n - 1 );
}
void impresion( int *v, int li, int ls )
{   int i;
// Impresion de resultados.
    cout << "\tli = " << li << "; ls = " << ls << "\n";
    cout << "  v = { " << v[li];
    for ( i = li + 1; i < ls; i++ )
    {     if ( i - li > 0 && (i - li )%15 == 0 ) cout << ",\n\t"
<< v[i];
            else cout << ", " << v[i];  }
    cout << " }\n\n";
    system ( "pause" );     }
```

14. Ejercicio: 8.6. Pag. 135: C Algoritmos, programacion y estructura de datos - Joyanes - Fernandez - Sanchez – Zahonero Escribir un programa. en lenguaje C/C++, que tenga como entrada una secuencia de numeros enteros positivos(mediante una variable entera). El programa debe hallar la suma de los digitos de cada entero y encontrar cual es el entero cuya suma de digitos es mayor. La suma de digitos ha de ser con una funcion recursiva. Observacion: El programa lee numeros enteros. Finaliza cuando lee un numero que no lo es.

```
#include<iostream>
using namespace std;
int sumadig( int nro );
int main()
{      int n, max = 0, sum, sumMax = 0;
/*  Variables del programa.
       Entrada:       n      : numero entero a considerar.
      Salida:      max    : numero entero cuya suma de digitos es
la mayor en el conuunto.
             sumMax: suma de los digitos de max  */
       do {       cout << "\nDigite un numero entero ";
             cin >> n;
             if ( n > 0 )
             {     sum  = sumadig( n );
                   if ( sum > sumMax )
                   {     sumMax = sum;
                         max = n;            }       }
       }  while ( n > 0 );
       cout << "\n\tEl numero seleccionado es " << max;
       cout << "\n\tLa suma de sus digitos es " << sumMax;
       cout << "\n";
       system ( "pause" );
}
int sumadig( int nro )
```

```
{       int div = 10;
/*  Varialbes de la funcion:
      Entrada:
           nro : numero a analizar.
      Valor retornado:
           Suma de digitos del numero (recursivamente)
      Auxiliar:
           div : unidad seguida de ceros,  inicialmente
inmediato mayor que nro.  */
      if ( nro < 10 ) return nro;
      while ( nro > div ) div *= 10;
      div /= 10;
    return ( nro/div + sumadig( nro - (nro/div)*div ) );}
```

15. // Utilizacion de enumeraciones

```
#include<iostream>
using namespace std;
main()
{          int d;
           enum dias
                  { domingo = 1, lunes, martes, miercoles,
jueves, viernes, sabado };
           enum dias semana;
           cout << "\n\n\tDigite en dia ";
           cin >> d;
           switch ( d ) {   case domingo:    cout << "\tDia
domingo";          break;
                       case lunes  :    cout << "\tDia lunes";
        break;
                       case martes :    cout << "\tDia martes";
        break;
                       case miercoles:  cout << "\tDia
miercoles";              break;
                       case jueves:     cout << "\tDia jueves";
        break;
                       case viernes:    cout << "\tDia viernes";
        break;
                       case sabado:     cout <<  "\tDia sabado";
        break;
                       default:  cout << "\n\tDia Dia invalido";
           }
// Observese que ocurre si se imprimen los siquientes:
           cout << "\n\n\tDomingo = " << domingo << ", lunes = "
<< lunes;
           cout << ", martes = " << martes << ", miercoles = "
<< miercoles << ",";
           cout << "\n\tjueves = " << jueves << ", viernes = "
<< viernes;
           cout << ", sabado = " << sabado;
           cout << "\n\n";
           system ( "pause" );}
```

16. Uso de ?

```
#include<iostream>
using namespace std;
main()
{   int n;
   float x;
   cout << "Digite un numero entero ";
   cin >> n;
   x = n > 0 ? 1 : 2;
   if ( x == 1 )      cout << "\n\nEl valor dado de n es
positivo";
      else cout << "\n\nEl valor dado de n es negativo o nulo";
   cout << "\n\n";
// Observe lo que sigue
   x = n > 0;
   if ( x )      cout << "\n\nEl valor dado de n es positivo";
      else cout << "\n\nEl valor dado de n es negativo o nulo";
   cout << "\n\n";
   system("pause");}
```

17. Uso de punteros y vectores

```
#include<iostream>
using namespace std;
int main()
{  int i, n;
   char *p;        // Declaracion de puntero
   do {
      cout << "\n\t\tDigite el numero de componentes del vector
";
      cin >> n;
   }  while( n < 2 );
   char v[n];      // Vector de caracteres
   for ( i = 0; i < n; i++ )
   {  cout << "\tDigite v[" << i << "]: ";
      cin >> v[i];    }
// Impresion de v, utilizando el  nombre del vector
   cout << "\n\nv = { " << v[0];
   for ( i = 1; i < n; i++ )
      cout << ", " << v[i];
   cout << " }";
// Impresion de v, utilizando el  puntero p
   p = v;
   cout << "\n\nv = { " << *p;
   for ( i = 1; i < n; i++ )
      cout << ", " << *(p + i);
   cout << " }";
// Impresion de las componentes, a partir de uno determinado
```

```
   cout << "\n\nv = { " << p;
   for ( i = 1; i < n; i++ ) cout << ", " << p+i;
// Impresion de las direcciones de memoria inicial ocupada por v
   cout << "\n\nVer &p  = { " << &p;
   cout << " }\n\n";
   cout << "\n\nVer &v = { " << &v;
   cout << " }\n\n";
   cout << "\n\nVer p  = { " << p;
   cout << " }\n\n";
   cout << "\n\nVer v = { " << v;
   cout << " }\n\n";
     cout << "\n\n &v = { " << &v[0];
   for ( i = 1; i < n; i++ )
      cout << ", " << &v[i];
   cout << " }\n\n";
   system ( "pause" );}
```

18. Uso de sizeof()

```
#include<iostream>
using namespace std;
int main()
{      cout << "\nPlataforma de " << 8*sizeof(int) << "
bits\n\n";
       system ( "pause" );}
```

19. Uso de estructuras REGISTRO DE EMPLEADOS

```
#include<iostream>
using namespace std;
int main()
{  int i, n;
   do {
      cout << "\n\t\tDigite el numero de empleados ";
      cin >> n;
   }  while( n < 2 );
   struct varEst       // Declaracion de la estructura
   {  long int DocId;
      char nom[30];
      double salario;   };
   struct varEst empl[10];
   for ( i = 0; i < n; i++ )
   {  cout << "\tDigite DocId[" << i << "]: ";
      cin >> empl[i].DocId;
      cout << "\tDigite Nombre[" << i << "]: ";
      cin >> empl[i].nom;
      cout << "\tDigite Salario[" << i << "]: ";
      cin >> empl[i].salario;   }
// Impresion de empl
   cout << "\n\nDocumento   Nombre y Apellido    Salario";
   for ( i = 0; i < n; i++ )
```

```
   {  cout << "\n" << empl[i].DocId;
      cout << ", " << empl[i].nom;
      cout << ", " << empl[i].salario;   }
   cout << " }\n\n";
   system ( "pause" );}
```

20. Uso de estructuras OPERACIONES CON FRACCIONES

```
#include<iostream>
#include<stdio.h>
using namespace std;
int lect();
int mcd( int a, int b );
struct fraccion
{     int numer, deno;  };
fraccion suma( fraccion f1, fraccion f2 );
fraccion prod( fraccion f1, fraccion f2 );
int main()
{  int aux, ope = 1;
   fraccion f1, f2, f;
   char msg[][20] = {"de la suma", "de la diferencia", "del
producto", "del cociente" };
   while ( ope )
   {  cout << "\nNumerador de la primera fraccion";
         f1.numer = lect();
         cout << "\nDenominador de la primera fraccion";
         f1.deno = lect();
         cout << "\nNumerador de la segunda fraccion";
         f2.numer = lect();
         cout << "\nDenominador de la segunda fraccion";
         f2.deno = lect();
         cout << "\n\t\t1. Suma de fracciones";
         cout << "\n\t\t2. Diferencia de fracciones";
         cout << "\n\t\t3. Producto de fracciones";
         cout << "\n\t\t4. Cociente de fracciones\n";
         cout << "\n\t\t0. Fin del programa\n";
         ope = lect();
         switch(ope)
         {
            case  1:  f = suma( f1, f2 );
                         break;
            case  2:  f2.numer = -f2.numer;
                         f = suma( f1, f2 );
                         break;
            case  3:  f = prod( f1, f2 );
                         break;
            case  4:  aux = f2.numer;
                         f2.numer = f2.deno;
                         f2.deno = aux;
                         f = prod( f1, f2 );
                         break;
            default:  cout << "\nOperacion no valida";
         }
```

```
// Impresion del resultado
           if ( ope == 1 || ope == 2 || ope == 3 || ope == 4 )
           {     if ( f.deno < 0 )
                 {     f.deno = -f.deno;
                       f.numer = -f.numer;   }
           cout << "\n\n\tRESULTADO " << msg[ope - 1] << ": " <<
f.numer << "/" << f.deno;   }
        cout << "\n\nContinua? (0 para finalizar )";
        cin >> ope;
        system ( "cls" );    }}
int lect() // Lectura de numero entero
{     float nro;
      do {
           cout << "\n\tDigite un numero entero y positivo ";
           cin >> nro;
      }  while ( nro < 0 || nro != (int)nro );
      return (int)nro;
}
int mcd( int a, int b ) // Halla el mcd de a y b
{     int p, q, r;
      p = a < b ? b : a;
      q = a > b ? b : a;
      r = p%q;
      while ( r )
      {     p = q;      q = r;
            r = p%q;    }
      return q;  }
fraccion suma( fraccion f1, fraccion f2 )
{  int aux;
      fraccion f;
   f.deno = f1.deno*f2.deno/mcd(f1.deno, f2.deno);
   f.numer = f.deno/f1.deno*f1.numer + f.deno/f2.deno*f2.numer;
   aux = mcd( f.numer, f.deno );

   return f;  }
fraccion prod( fraccion f1, fraccion f2 )
{  int aux;
      fraccion f;
   f.numer = f1.numer*f2.numer;
   f.deno = f1.deno*f2.deno;
   aux = mcd( f.numer, f.deno );

   return f; }
```

21. Uso de switch

```
#include<iostream>
using namespace std;
main()
{
    int n;
    cout << "\n\n\t\t Menu Principal";
    cout << "\n\n\t1. Realizar la tarea uno";
    cout << "\n\t2. Realizar la tarea dos";
    cout << "\n\t3. Realizar la tarea tres";
    cout << "\n\n\t0. Salir del sistema";
    cout << "\n\n\t\tDigite un numero entero ";
    cin >> n;
    switch(n)
    {          case  1:    cout << "\nDebo realizar la tarea
uno";   break;
          case  2:    cout << "\nDebo realizar la tarea dos";
break;
          case  3:    cout << "\nDebo realizar la tarea tres";
break;
          case  0:    cout << "\nMe retiro a descanzar";
break;
          default:    cout << "\nNo es una opcion valida";    }
    cout << "\n\n\t";
    system("pause");}
```

22. Uso de system

```
#include<iostream>
#include<stdio.h>
#include<string.h>
using namespace std;
int main()
{    int i, recnum = 0;
      char registro[128], *regaux, nombre[32],
archivos[100][32];
      FILE *pf;
      system("dir > archivos.txt ");
      pf = fopen ( "archivos.txt", "r" );
      if ( pf == NULL )
           cout << "\n\tError al abrir el achivo\n";
      else
      {       while ( !feof( pf ) )
       {      fgets( registro, 128, pf );
           recnum++;
           if ( recnum > 7 && !strstr( registro, "bytes" ) )
           {                    regaux = registro;
                     strcpy ( nombre, ( registro + 36 ) );
                     strcpy( archivos[recnum - 8], nombre );
```

```
                    }       }       }
    cout << "\n";
    for ( i = 0; i < recnum - 10; i++ )
    {       cout << "Archivo[" << i << "] : " << archivos[i];
      if ( i && i%20 == 0 )
            {     cout << "\n";
                  system ( "pause" );
                  system ( "cls" );
                  cout << "\n";         }     }
      cout << "\n";
      system("pause");}
```

23. Uso de union

```
#include<iostream>
#include<string.h>
using namespace std;
main()
{
          int d;
          char entrada[30];
          union nombres
          {  char nomCompl[30];
               char apell[20];
               char nom;  };
          union nombres ApeNom;
          cout << "\n\n\tDigite el nombre comleto (sin
espacios) ";
          cin >> entrada;
          strcpy( ApeNom.nomCompl, entrada );
          cout << "\n\n" << ApeNom.nomCompl;
          cout << "\n" << ApeNom.apell;
          cout << "\n" << ApeNom.nom;
          cout << "\n\n";
          system ( "pause" );}
```

Bibliografía

C. Algoritmos, programación y estructuras de datos, L. Joyanes y otros, Serie Schaun, Editorial McGraw-Hill/Interamericana. 2005

http://correo.uan.edu.mx/~iavalos/introprog.htm

http://www.uhu.es/04004/material/

http://c.conclase.net